

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN**



**BÁO CÁO ĐỒ ÁN MÔN HỌC
Toán ứng dụng và thống kê**

Giáo viên hướng dẫn: ThS. Lê Nhật Nam
ThS. Võ Nam Thục Đoan

Nhóm thực hiện: Nhóm 3 - Lớp 24CTT2
Thành viên: 24120018 – Đào Thị Quỳnh Anh
24120347 - Phan Lê Đăng Khoa
24120348 – Hà Đăng Khôi
24120352 – Lương Trung Kiên
24120418 – Nguyễn Minh Quân

Hồ Chí Minh, 2026

Mục lục

Bảng phân công công việc	7
0 Tổng quan Kiến trúc Hệ thống và Các Module dùng chung	10
0.1 Tổ chức file theo hướng Module hóa	10
0.2 Module cấu hình toàn cục: config.py	11
0.3 Các hàm tiện ích nội bộ dùng chung: utils.py	12
0.4 Module tiện ích kiểm thử dùng chung: test_utils.py	15
0.5 Quy ước Import và Chạy Test	16
0.6 Quản lý Sai số Dấu phẩy động: IEEE 754 và Machine Epsilon	17
0.6.1 Giới hạn biểu diễn: Machine Epsilon	17
0.6.2 Chiến lược quản lý sai số trong dự án	17
0.6.3 Ảnh hưởng của điều kiện số lên OLS	18
1 Phần 1: Lý Thuyết Data Fitting và Minh Họa	19
1.1 Bài Toán Data Fitting	19
1.1.1 Phát Biểu Bài Toán Tổng Quát	19
1.1.2 Các Giả Thiết Gauss–Markov	19
1.2 Phương Pháp Ordinary Least Squares (OLS)	20
1.2.1 Hàm Mất Mát và Nghiệm OLS	20
1.2.2 Ma Trận Chiều và Hat Matrix	21
1.2.3 Định Lý Gauss–Markov	21
1.2.4 Ước Lượng Phương Sai Nhiễu	22
1.3 Đánh Giá Mô Hình	23
1.3.1 Hệ Số Xác Định R^2 và $\bar{R}^2$ Hiệu Chỉnh	23
1.3.2 Kiểm Định Giả Thuyết	23
1.4 Các Vấn Đề Nâng Cao	24
1.4.1 Đa Cộng Tuyến và Hệ Số VIF	24
1.4.2 Hồi Quy Ridge và Lasso (Regularization)	25
1.5 Phân Tích Phần Dư	26
1.6 Cross-Validation và Lựa Chọn Mô Hình	27
1.6.1 k -Fold Cross-Validation	27
1.6.2 Tiêu Chí Lựa Chọn Mô Hình	29
1.7 Kết Quả Cài Đặt và Kiểm Chứng	29
1.7.1 Tổng Hợp Các Hàm Cài Đặt	29
1.7.2 Cấu Trúc Phụ Thuộc Giữa Các Hàm	30

1.7.3	Kiểm Chứng Với NumPy và Sklearn	30
1.7.4	Nhận Xét Kết Quả Phần 1	31
2	Phần 2: Ứng Dụng Data Fitting vào Dữ Liệu Thực Tế	33
2.1	Mô tả Bộ Dữ Liệu	33
2.1.1	Nguồn dữ liệu và bài toán hồi quy	33
2.1.2	Lý do chọn bộ dữ liệu	34
2.2	Khám Phá Dữ Liệu Ban Đầu (EDA)	34
2.2.1	Missing values trước xử lý	36
2.3	Thiết Kế Pipeline Tiền Xử Lý	37
2.3.1	Nguyên tắc tránh data leakage	37
2.3.2	Cơ sở toán học và vật lý của phép biến đổi Logarit	38
2.3.3	Log-transform các biến lệch mạnh	39
2.3.4	Xử lý ngoại lai bằng Winsorization	41
2.3.5	Xử lý missing values bằng MICE	43
2.3.6	Loại biến bằng VIF	45
2.3.7	Chuẩn hóa dữ liệu	46
2.4	Xây Dựng và So Sánh Mô Hình	47
2.4.1	Các mô hình chính	47
2.4.2	OLS full và chọn biến theo p-value	47
2.4.3	Chọn λ cho Ridge và Lasso	47
2.5	Kết Quả Thực Nghiệm	48
2.5.1	So sánh mô hình trên test set	48
2.5.2	Phân tích hệ số và feature importance	49
2.5.3	Actual vs Predicted và phân tích phần dư	50
2.6	Phương Pháp Nâng Cao: Kernel Ridge Regression	51
2.7	Kết Luận Phần 2	52
3	Kết luận	53
3.1	Tóm tắt kết quả đạt được	53
3.2	Bài học rút ra	54
3.3	Khó khăn chuyên môn và các nghịch lý Toán học tính toán	54

Danh sách hình vẽ

1	Phân phối $\hat{\beta}_{OLS}$ (xanh) và $\tilde{\beta}_{alt}$ (cam) qua 1000 mô phỏng. Đường đỏ dọc là giá trị β thực. OLS tập trung xung quanh β thực (không chệch) với độ phân tán thấp hơn estimator thay thế (phương sai nhỏ hơn).	22
2	Ridge Trace: giá trị các hệ số hồi quy theo λ (log scale). Khi $\lambda \rightarrow 0$, các hệ số tiệm cận nghiệm OLS; khi $\lambda \rightarrow \infty$, tất cả hệ số co về 0.	26
3	λ vs CV-MSE (5-fold). Đường đỏ đứt là λ^* tối ưu. Vùng λ quá nhỏ: mô hình overfit (MSE cao trên validation); λ quá lớn: underfit do shrinkage quá mạnh.	26
4	Bốn biểu đồ chẩn đoán phần dư. Trên dữ liệu giả lập thỏa mãn GM1–GM5: (1) phần dư phân tán đều, (2) Q-Q plot gần tuyến tính, (3) phương sai đồng đều, (4) Cook's Distance không có điểm nào vượt ngưỡng $4/n$	28
5	Sơ đồ phụ thuộc giữa các hàm Phần 1. Màu xanh là các module cơ sở (utils.py, config.py); mũi tên thể hiện hàm gọi/phụ thuộc vào hàm khác.	31
6	Histogram các biến số trước tiền xử lý. Một số biến như pl_orbper, pl_trandep, pl_insol có phân phối lệch mạnh, vì vậy cần log-transform trước khi hồi quy.	35
7	Ma trận tương quan Pearson giữa các biến số. Hình này được dùng để nhận diện các nhóm biến dễ gây đa cộng tuyến, đặc biệt là nhóm quỹ đạo (pl_orbper, pl_orbsmax) và nhóm nhiệt độ/năng lượng (pl_eqt, pl_insol, st_teff).	36
8	Scatter plot của các biến có tương quan cao nhất với target pl_rade. Hình này giúp đánh giá sơ bộ tính tuyến tính và mức độ nhiễu trước khi xây dựng mô hình.	36
9	So sánh phân phối trước và sau log-transform. Các biến có đuôi phải dài như pl_orbper và pl_insol được nén thang đo rõ rệt sau khi chuyển sang log_*.	41
10	Boxplot trước và sau Winsorization. Các giá trị cực trị ở hai đuôi được kéo về ngưỡng 1% và 99%, trong khi phần trung tâm của phân phối gần như được giữ nguyên.	42
11	So sánh phân phối observed và phần được MICE impute ở các cột thiếu nhiều nhất. Giá trị impute không bị gom về một điểm trung vị duy nhất mà vẫn có độ phân tán tự nhiên.	44
12	VIF trước và sau khi thanh lọc đa cộng tuyến. Trước khi lọc, nhiều biến vượt ngưỡng 10; sau khi loại log_pl_orbper và log_pl_eqt, các VIF còn lại đều nằm dưới ngưỡng.	46
13	So sánh Test R^2 , RMSE và MAE giữa các mô hình. Lasso đạt Test R^2 cao nhất, nhưng khác biệt giữa Lasso, OLS full và Ridge khá nhỏ.	49
14	Feature importance dựa trên hệ số Ridge sau chuẩn hóa. Các hệ số được so sánh trên cùng thang đo vì pipeline đã chuẩn hóa z-score.	49

15	Giá trị thực tế so với giá trị dự đoán trên tập test của mô hình tốt nhất. Các điểm càng gần đường chéo thì mô hình dự đoán càng tốt.	50
16	Bốn biểu đồ chẩn đoán phần dư của mô hình tốt nhất: Residuals vs Fitted, Normal Q-Q, Scale-Location và Cook's Distance.	51

Danh sách bảng

1	Bảng phân công công việc và mức độ đóng góp	7
2	Tổng hợp các hàm đã cài đặt trong Phần 1	30
3	Kết quả kiểm chứng F1 và F6 với NumPy/sklearn	31
4	Các cột có missing values nhiều nhất trước MICE imputation trên tập train . . .	37
5	Cơ sở toán học và vật lý cho các biến được log-transform	39
6	Ảnh hưởng của log-transform lên skewness	40
7	Tóm tắt Winsorization trên tập train	42
8	Tóm tắt giá trị được MICE impute trên tập train	44
9	Các vòng lọc VIF	45
10	Một số VIF lớn nhất sau khi thanh lọc đa cộng tuyến	45
11	Một số hệ số OLS full có ý nghĩa thống kê	47
12	Một số kết quả Ridge CV theo λ	48
13	Kết quả Lasso CV theo λ	48
14	So sánh các mô hình trên tập test	48

Danh mục mã nguồn

1	Module <code>config.py</code> — hằng số và hàm xử lý sai số dùng chung	12
2	Trích đoạn <code>utils.py</code> : các hàm cốt lõi cho OLS	14
3	Lệnh chạy toàn bộ unit test từ thư mục gốc	16
4	Hàm <code>ols_fit</code> - tính nghiệm OLS từ công thức Normal Equations	20
5	Hàm <code>coef_inference</code> - tính standard error, t-statistic, p-value và khoảng tin cậy 95%	24
6	Hàm <code>vif</code> - tính VIF cho từng biến, dùng F1 và F3	24
7	Hàm <code>kfold_cv</code> - thực hiện k-fold cross-validation, dùng được với <code>ols_fit</code> , <code>ridge_fit</code> , <code>lasso_fit</code>	27
8	Tách train/test trước khi fit preprocessing	37

BẢNG PHÂN CÔNG CÔNG VIỆC VÀ MỨC ĐỘ ĐÓNG GÓP

Bảng 1: Bảng phân công công việc và mức độ đóng góp

STT	Họ tên / MSSV	Nhiệm vụ đảm nhiệm chi tiết	Đóng góp
1	24120018 Đào Thị Quỳnh Anh	• Phần 1: Cài đặt thuật toán hồi quy có cực tiểu hóa hình phạt: Ridge Regression (nghiệm dạng đóng) và Lasso Regression (giải bằng phương pháp Coordinate Descent). • Phần 1: Xây dựng hệ thống chẩn đoán mô hình trực quan (vẽ 4 biểu đồ phần dư) và cài đặt thuật toán kiểm chứng chéo K-Fold Cross-Validation từ đầu để tinh chỉnh siêu tham số λ. • Phần 2 (T7 & QA): Cài đặt mô hình nâng cao (Kernel Ridge Regression hoặc Bayesian Linear Regression) để so sánh đối chiếu. Thực hiện rà soát chất lượng mã nguồn (Quality Assurance), kiểm tra Unit Test và đảm bảo tính nhất quán của hằng số RANDOM_STATE.	100%
2	24120347 Phan Lê Đăng Khoa	• Phần 1: Lập trình lõi thuật toán Hồi quy tuyến tính đa biến (OLS) bằng phương trình chuẩn (Normal Equations) sử dụng Python thuần. • Phần 1: Tính toán ma trận chiếu (Hat Matrix), kiểm chứng tính chất đại số tuyến tính (lũy đẳng, đối xứng) và trích xuất trị riêng. Cài đặt hệ thống đo lường với các chỉ số thống kê cốt lõi (R^2, Adj-R^2, RSS, F-statistic). • Phần 2 (T3): Xây dựng lớp DataPipeline hướng đối tượng nhằm tự động hóa các khâu: nội suy KNN, xử lý ngoại lai (Winsorization), chuẩn hóa (Scaling) và mã hóa (Encoding), đảm bảo tuyệt đối không xảy ra rò rỉ dữ liệu (No Data Leakage).	100%

STT	Họ tên / MSSV	Nhiệm vụ đảm nhiệm chi tiết	Đóng góp
3	24120348 Hà Đăng Khôi	• Cơ sở hạ tầng: Khởi tạo kiến trúc dự án (README.md, requirements.txt) và thiết kế Jupyter Notebook trình bày cơ sở lý thuyết Toán học cho các thuật toán từ F1 đến F5. • Phần 2 (T1, T2): Chọn lọc và tải tập dữ liệu thực tế. Thực hiện Khảo sát dữ liệu toàn diện (EDA): báo cáo dữ liệu khuyết thiếu, phân tích phân phối (Histogram, Boxplot), và tính toán ma trận tương quan (Heatmap). • Phần 2 (T6): Xây dựng thuật toán nhận diện điểm ngoại lai bằng phương pháp IQR. Trực quan hóa mức độ quan trọng của các biến độc lập (Feature Importance) bằng Bar chart và tổng hợp báo cáo Notebook.	100%
4	24120352 Lương Trung Kiên	• Cơ sở hạ tầng: Thiết lập khung báo cáo $\text{\LaTeX}$ chuẩn học thuật. Thiết kế Jupyter Notebook minh họa lý thuyết phần chẩn đoán và xác thực chéo (các hàm từ F6 đến F10). • Phần 2 (T5): Lập bảng tổng hợp, so sánh hiệu năng các mô hình trên tập kiểm thử thông qua độ đo MAE, RMSE, R^2. Phân tích chuyên sâu biểu đồ phần dư của mô hình tối ưu. • Báo cáo: Đưa ra các nhận định đánh giá giả định Gauss-Markov dựa trên số liệu. Chịu trách nhiệm viết phần Kết luận, căn chỉnh định dạng $\text{\LaTeX}$, xử lý caption hình ảnh và biên soạn tài liệu tham khảo theo chuẩn BibTeX.	100%

STT	Họ tên / MSSV	Nhiệm vụ đảm nhiệm chi tiết	Đóng góp
5	24120418 Nguyễn Minh Quân	• Phần 1: Cài đặt module suy diễn thống kê: tính toán sai số chuẩn, t-statistic, p-value và khoảng tin cậy 95% cho các hệ số hồi quy. • Phần 1: Cài đặt thuật toán tính Hệ số nhân phương sai (VIF) nhằm phát hiện đa cộng tuyến. Lập trình mô phỏng Monte Carlo ($N = 1000$) để kiểm chứng thực nghiệm định lý Gauss-Markov. • Phần 2 (T4): Sử dụng Pipeline để huấn luyện và đánh giá 4 mô hình (OLS cơ bản, OLS chọn lọc biến, Ridge, Lasso) trên tập dữ liệu thực. Triển khai các bài kiểm định thống kê chuẩn mực (Shapiro-Wilk, Breusch-Pagan).	100%

0 Tổng quan Kiến trúc Hệ thống và Các Module dùng chung

Trước khi đi vào chi tiết của từng phần, mục này trình bày kiến trúc tổng thể của toàn bộ dự án — cách các module Python được tổ chức, các thành phần hạ tầng dùng chung giữa Phần 1 (lý thuyết và minh họa) và Phần 2 (ứng dụng thực tế), và nền tảng thiết kế về quản lý sai số số học mà toàn bộ codebase đều tuân theo. Việc hiểu rõ kiến trúc này giúp tránh trùng lặp mã, đảm bảo tính nhất quán khi kiểm chứng kết quả, và tạo điều kiện để hai phần của dự án chia sẻ cùng một pipeline hạ tầng.

0.1 Tổ chức file theo hướng Module hóa

Toàn bộ dự án được thiết kế theo nguyên tắc **module hóa** (modular design): mỗi file Python đảm nhiệm một trách nhiệm duy nhất và rõ ràng, hạn chế phụ thuộc vòng (circular dependency), đồng thời cho phép tái sử dụng các thành phần cốt lõi mà không phải viết lại mã. Cụ thể, dự án được chia thành bốn nhóm chức năng lớn:

- **Hạ tầng dùng chung (root):** `config.py` và `utils.py` nằm ở thư mục gốc. Đây là các module không phụ thuộc vào bất cứ module nào khác trong dự án; mọi module cấp trên đều có thể import từ đây. Hai module này cung cấp hằng số toàn cục, phép toán đại số tuyến tính từ đầu, và hàm xây dựng null vector phục vụ demo Gauss–Markov.
- **Tiện ích kiểm thử dùng chung:** `test_utils.py` cũng nằm ở thư mục gốc, cung cấp các hàm sinh dữ liệu giả lập (`make_linear_data`, `make_collinear_data`), các hàm assert có log màu (`assert_close`, `assert_shape`, v.v.) và các hàm kiểm chứng với sklearn (`verify_vs_sklearn_ols`, `verify_vs_sklearn_ridge`). Tất cả file unit test trong dự án import từ đây.
- **Module nghiệp vụ Phần 1 (part1/):** Gồm các file cài đặt thuật toán OLS và các phương pháp liên quan — `ols_implementation.py` (F1–F5), `ridge_lasso.py` (F6–F7), `residual_analysis.py` (F8), `cross_validation.py` (F9), `gauss_markov_demo.py` (F10) — cùng file kiểm thử riêng `test_ols_implementation.py`.
- **Module nghiệp vụ Phần 2 (part2/):** Gồm `data_pipeline.py` (pipeline tiền xử lý dữ liệu thực), `model_comparison.py` (so sánh ≥ 3 mô hình trên test set), và `advanced_methods.py` (Kernel/Bayesian, tùy chọn). Các module này tái sử dụng trực tiếp F1, F6, F7, F8, F9 từ part1/ thay vì viết lại.

Cấu trúc thư mục đầy đủ của dự án như sau:

```

Project2_DataFitting/
|-- config.py           # Hằng số toàn cục (EPSILON, RANDOM_STATE)
|-- utils.py           # Phép toán đại số tuyến tính tu đầu
|-- test_utils.py      # Tiện ích kiểm thử dùng chung
|-- part1/
|   |-- ols_implementation.py  # F1-F5: OLS, Hat matrix, VIF, ...
|   |-- ridge_lasso.py        # F6-F7: Ridge, Lasso, Trace
|   |-- residual_analysis.py  # F8: 4 biểu đồ chẩn đoán
|   |-- cross_validation.py   # F9: k-Fold CV
|   |-- gauss_markov_demo.py  # F10: Monte Carlo demo
|   '-- test_ols_implementation.py
|-- part2/
|   |-- data/
|   |   '-- <ten_dataset>.csv
|   |-- data_pipeline.py      # Tiền xử lý, encoding, chuẩn hóa
|   |-- model_comparison.py   # So sánh mô hình trên test set
|   '-- advanced_methods.py   # Kernel / Bayesian (tùy chọn)
|-- report/
|   '-- report.pdf
'-- requirements.txt

```

Cấu trúc này đảm bảo: khi một module lỗi (ví dụ `utils.py`) được cập nhật, toàn bộ codebase hưởng ngay lợi ích mà không cần sửa lại ở nhiều nơi. Đồng thời, Phần 2 không cần cài đặt lại OLS hay Ridge — các hàm từ Phần 1 được import trực tiếp, đảm bảo kết quả nhất quán giữa hai phần.

0.2 Module cấu hình toàn cục: `config.py`

Module `config.py` là file cấu hình dùng chung đầu tiên cần được hiểu. Nó định nghĩa các hằng số và hàm tiện ích được sử dụng xuyên suốt toàn bộ dự án, từ các phép tính đại số trong `utils.py` đến quá trình shuffle trong `cross_validation.py`.

- `EPSILON = 1e-12`: Ngưỡng epsilon toàn cục. Mọi số có giá trị tuyệt đối nhỏ hơn ngưỡng này đều được coi là bằng 0 về mặt số học. Ngưỡng này được dùng nhất quán trong tất cả phép chia, kiểm tra pivot, và tính VIF để tránh chia cho 0.
- `RANDOM_STATE = 42`: Seed toàn cục đảm bảo tính tái lập (*reproducibility*) của toàn bộ dự án — từ sinh dữ liệu giả lập trong `test_utils.py`, Fisher-Yates shuffle trong `cross_validation.py`, đến LCG random generator trong `gauss_markov_demo.py`.
- `is_zero(x)`: Kiểm tra xem một số x có xấp xỉ bằng 0 hay không theo ngưỡng `EPSILON`.
- `zero_rectify(value)`: Nếu `value` đủ nhỏ thì trả về 0.0 thay vì giữ nguyên giá trị nhiễu số.

học (ví dụ: $-2.77e-16$), tránh hiện tượng “ô nhiễm” kết quả qua nhiều bước tính toán.

- `calculate_relative_error(A, x_hat, b)`: Tính sai số tương đối $\|A\hat{x} - b\|_2 / \|b\|_2$ — dùng trong unit test để kiểm tra chất lượng nghiệm OLS và Ridge.

```

1 import math
2
3 EPSILON: float = 1e-12
4 RANDOM_STATE: int = 42
5
6 def is_zero(x: float) -> bool:
7     """Kiểm tra một số có xấp xỉ bằng 0 hay không."""
8     return abs(x) < EPSILON
9
10 def zero_rectify(value: float) -> float:
11     """Khu giá trị về 0 nếu nó đủ nhỏ."""
12     return 0.0 if is_zero(value) else value
13
14 def calculate_relative_error(A, x_hat, b) -> float:
15     """Tính sai số tương đối ||A*x_hat - b||_2 / ||b||_2."""
16     n = len(b)
17     residual = [sum(A[i][j]*x_hat[j] for j in range(len(x_hat)))-b[i]
18                 for i in range(n)]
19     norm_r = math.sqrt(sum(r*r for r in residual))
20     norm_b = math.sqrt(sum(bi*bi for bi in b))
21     if is_zero(norm_b):
22         return 0.0 if is_zero(norm_r) else float('inf')
23     return norm_r / norm_b

```

Đoạn mã 1: Module `config.py` — hằng số và hàm xử lý sai số dùng chung

Việc tách riêng hằng số và hàm xử lý sai số vào `config.py` giúp đảm bảo tính nhất quán: giá trị ngưỡng `EPSILON` chỉ cần khai báo một lần duy nhất. Đặc biệt với dự án này, `RANDOM_STATE` đóng vai trò quan trọng hơn so với các dự án tính toán số thuần túy, vì kết quả cross-validation và demo Gauss–Markov phụ thuộc trực tiếp vào thứ tự ngẫu nhiên của dữ liệu.

0.3 Các hàm tiện ích nội bộ dùng chung: `utils.py`

Module `utils.py` cung cấp một thư viện các phép toán đại số tuyến tính cơ bản được tái sử dụng xuyên suốt dự án. Toàn bộ được cài đặt **từ đầu bằng Python thuần**, không phụ thuộc vào `numpy` hay bất kỳ thư viện số học nào. Đây là yêu cầu bắt buộc của đề bài — các hàm này là “động cơ” của toàn bộ cài đặt OLS, Ridge, Lasso, và Hat Matrix.

Nhóm phép toán ma trận cơ bản

- `identity_matrix(n)`: Tạo ma trận đơn vị I_n dưới dạng list 2 chiều.

- `transpose(A)`: Chuyển vị ma trận bằng `zip(*A)`, trả về $\mathbf{A}^T$. Được dùng trong mọi lần tính $\mathbf{X}^T \mathbf{X}$ và $\mathbf{X}^T \mathbf{y}$.
- `matmul(A, B)`: Nhân hai ma trận theo thuật toán $\mathcal{O}(mnp)$. Có tối ưu nhỏ: bỏ qua vòng lặp trong nếu $a_{ik} \approx 0$ (kiểm tra qua `is_zero`), giảm phép tính không cần thiết trên ma trận thưa.
- `matvec(A, v)`: Tích ma trận–vector $\mathbf{A}\mathbf{v}$, trả về list 1 chiều. Được dùng để tính $\hat{\mathbf{y}} = \mathbf{X}\hat{\boldsymbol{\beta}}$ trong mọi hàm fit.
- `dot_product(u, v)`: Tích vô hướng $\mathbf{u}^T \mathbf{v}$. Được dùng trong Gram–Schmidt và coordinate descent của Lasso.
- `vector_norm(v)`: Chuẩn Euclidean $\|\mathbf{v}\|_2$; dùng `max(0.0, ...)` để phòng kết quả âm nhỏ do sai số số học trước khi lấy căn.
- `normalize(v)`: Chuẩn hóa vector về đơn vị ($\|\mathbf{v}\|_2 = 1$); trả về vector $\mathbf{0}$ nếu chuẩn ban đầu xấp xỉ 0.
- `add_bias(X)`: Thêm cột hằng số 1 vào đầu ma trận $\mathbf{X}$ tạo ra ma trận design $[\mathbf{1} \mid \mathbf{X}]$. Được gọi đầu tiên trong `ols_fit`, `hat_matrix`, `coef_inference` và các hàm inference khác.

Nhóm giải hệ tuyến tính

- `solve_system(A, b)`: Giải hệ $\mathbf{Ax} = \mathbf{b}$ bằng phép khử Gauss với *partial pivoting* và back-substitution. Ném `ValueError` khi ma trận suy biến ($\text{pivot} < 10^{-14}$). Đây là hàm cốt lõi nhất — được gọi bởi `ols_fit` (giải Normal Equations) và `ridge_fit` (giải hệ Ridge đã điều chỉnh).
- `inverse(A)`: Tính $\mathbf{A}^{-1}$ bằng phương pháp Gauss–Jordan trên ma trận mở rộng $[\mathbf{A} \mid \mathbf{I}]$. Được dùng trong `hat_matrix` (tính $(\mathbf{X}^T \mathbf{X})^{-1}$) và `coef_inference` (tính ma trận hiệp phương sai $\hat{\sigma}^2(\mathbf{X}^T \mathbf{X})^{-1}$).

Nhóm kiểm tra và chuẩn hóa

- `is_upper_triangular(U, tol)`: Kiểm tra ma trận $\mathbf{U}$ có phải tam giác trên không.
- `check_identity(M, tol)`: Kiểm tra ma trận $\mathbf{M}$ có xấp xỉ ma trận đơn vị không. Dùng trong unit test để kiểm chứng $(\mathbf{X}^T \mathbf{X})(\mathbf{X}^T \mathbf{X})^{-1} \approx \mathbf{I}$.
- `rectify_matrix(M)`, `rectify_vector(v)`: Áp dụng `zero_rectify` lên từng phần tử. Dùng sau các phép nhân ma trận lớn để làm sạch nhiễu số học tích lũy.
- `max_abs_diff(A, B)`: Tính $\max_{ij} |A_{ij} - B_{ij}|$ — thước đo độ chính xác trong unit test, ví dụ kiểm tra $\mathbf{H}^2 \approx \mathbf{H}$ (tính idempotent của Hat Matrix).

Nhóm trực giao hóa và null space

- `orthogonalize(v, basis)`: Chiếu vector $\mathbf{v}$ ra khỏi tất cả các vector trong `basis` theo công thức Gram–Schmidt cổ điển:

$$\mathbf{w} = \mathbf{v} - \sum_{\mathbf{b} \in \text{basis}} \langle \mathbf{v}, \mathbf{b} \rangle \cdot \mathbf{b}. \quad (1)$$

Mỗi lần trừ xong một thành phần, kết quả được làm sạch qua `zero_rectify` để triệt tiêu nhiễu dấu phẩy động.

- `find_new_unit_vector(basis, dim)`: Khi cần bổ sung vector mới vào cơ sở trực giao, hàm lần lượt thử các vector đơn vị $\mathbf{e}_1, \mathbf{e}_2, \dots$. Mỗi ứng viên được trực giao hóa với basis rồi chuẩn hóa. Ném `ValueError` nếu thất bại.
- `build_null_vector(X_bias)`: Xây dựng vector $\mathbf{v} \in \text{Null}(\mathbf{X}^T)$, tức $\mathbf{X}^T \mathbf{v} \approx \mathbf{0}$, bằng cách tạo vector ngẫu nhiên (LCG reproducible) rồi chiếu ra khỏi không gian cột của $\mathbf{X}$. Được dùng duy nhất bởi `gauss_markov_demo.py` để xây dựng estimator thay thế chứng minh OLS là BLUE.

```

1 from config import is_zero, zero_rectify, EPSILON
2 import math
3
4 def add_bias(X: list[list[float]]) -> list[list[float]]:
5     """Thêm cột 1 (bias/intercept) vào đầu ma trận X."""
6     return [[1.0] + list(row) for row in X]
7
8 def solve_system(A, b) -> list[float]:
9     """Giải Ax = b bằng khu Gauss với partial pivoting."""
10    n = len(b)
11    M = [A[i][:] + [b[i]] for i in range(n)]
12    for col in range(n):
13        # Partial pivoting
14        pivot_row = max(range(col, n), key=lambda r: abs(M[r][col]))
15        M[col], M[pivot_row] = M[pivot_row], M[col]
16        if abs(M[col][col]) < 1e-14:
17            raise ValueError("Ma trận suy biến.")
18        for row in range(col + 1, n):
19            factor = M[row][col] / M[col][col]
20            for j in range(col, n + 1):
21                M[row][j] -= factor * M[col][j]
22    # Back-substitution
23    x = [0.0] * n
24    for i in range(n - 1, -1, -1):
25        x[i] = M[i][n]
26        for j in range(i + 1, n):
27            x[i] -= M[i][j] * x[j]
28        x[i] /= M[i][i]
29    return x
30
31 def build_null_vector(X_bias: list[list[float]]) -> list[float]:
32     """Xây dựng v thuộc Null(X^T): X^T v ≈ 0 (dùng cho GM demo)."""
33    n, p1 = len(X_bias), len(X_bias[0])
34    cols = [[X_bias[i][j] for i in range(n)] for j in range(p1)]
35    state = 12345

```

```

36     v = []
37     for _ in range(n):
38         state = (state * 1103515245 + 12345) & 0x7FFFFFFF
39         v.append((state / 0x7FFFFFFF) * 2.0 - 1.0)
40     for col in cols:                # Gram-Schmidt ra khỏi span(X)
41         c = dot_product(v, col) / max(dot_product(col, col), EPSILON)
42         v = [v[i] - c * col[i] for i in range(n)]
43     norm_v = math.sqrt(sum(vi*vi for vi in v))
44     return [vi / norm_v for vi in v]

```

Đoạn mã 2: Trích đoạn `utils.py`: các hàm cốt lõi cho OLS

0.4 Module tiện ích kiểm thử dùng chung: `test_utils.py`

Một điểm khác biệt đáng chú ý của dự án này so với dự án trước là sự xuất hiện của `test_utils.py` — một module kiểm thử dùng chung hoàn chỉnh. Module này phục vụ ba mục đích:

Sinh dữ liệu giả lập nhất quán

- `make_linear_data(n, beta, sigma, seed)`: Sinh dataset tuyến tính $y = X\beta + \varepsilon$ với $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, dùng `numpy.default_rng(seed)` để tái lập. Trả về (X, y) với X chưa có cột bias.
- `make_multifeature_data(n, p, sigma, seed)`: Sinh dataset nhiều biến với β ngẫu nhiên — dùng để kiểm tra tổng quát hơn.
- `make_collinear_data(n, seed)`: Sinh dataset có đa cộng tuyến: $x_3 \approx x_1 + x_2 + \varepsilon_{\text{small}}$. Dùng đặc biệt cho unit test F5 (vif) để xác nhận $VIF > 10$ được phát hiện đúng.

Toàn bộ file unit test của dự án đều dùng các factory này thay vì tự sinh data riêng. Điều này đảm bảo cùng một seed cho ra cùng một dữ liệu, giúp debug và so sánh kết quả giữa các thành viên nhóm.

Hàm assert có log màu Các hàm `assert_*` trong `test_utils.py` không ném exception mà trả về bool và in log màu qua `TestLogger`. Điều này cho phép chạy toàn bộ suite mà không dừng ở test đầu tiên thất bại:

- `assert_close(actual, expected, label, rtol, atol)`: So sánh hai giá trị/array với dung sai tương đối và tuyệt đối, dùng `numpy.testing.assert_allclose` nội bộ.
- `assert_equal(actual, expected, label)`: So sánh bằng tuyệt đối (`==`).
- `assert_true(condition, label, details)`: Kiểm tra điều kiện Boolean.
- `assert_shape(arr, expected_shape, label)`: Kiểm tra shape của array/list qua `numpy.asarray().shape`.
- `assert_in_range(val, lo, hi, label)`: Kiểm tra $lo \leq val \leq hi$.
- `assert_raises(exc_type, fn, *args, label)`: Kiểm tra hàm `fn` ném đúng loại exception. Phân biệt giữa “không ném exception” và “ném sai loại exception” — cả hai đều được

log riêng.

Hàm kiểm chứng với sklearn

- `verify_vs_sklearn_ols(X, y, beta_hat, rtol)`: So sánh `beta_hat` từ F1 với `sklearn.LinearRegression` — kiểm tra cả intercept lẫn coefficients.
- `verify_vs_sklearn_ridge(X, y, beta_hat, lam, rtol)`: So sánh với `sklearn.Ridge(alpha=lam)`. Dùng ngưỡng `rtol=1e-3` (lỏng hơn OLS) vì Ridge có chuẩn hóa có thể khác convention.
Cả hai hàm đều bỏ qua kiểm tra (trả về True với cảnh báo) nếu sklearn không được cài đặt, đảm bảo unit test vẫn chạy được trên môi trường không có sklearn.

0.5 Quy ước Import và Chạy Test

Để tránh lỗi import path giữa các môi trường khác nhau, dự án áp dụng quy ước thống nhất:

Quy tắc import path toàn dự án

- Tất cả file trong `part1/` import lẫn nhau không có prefix `part1..`

Dung - chạy tu thu muc part1/
`from ols_implementation import ols_fit, hat_matrix`
- Mỗi file đặt `sys.path.insert(0, os.path.dirname(__file__))` ở đầu để đảm bảo import đúng dù được gọi từ thư mục nào.
- Phần 2 import từ Phần 1 qua `sys.path.insert` trở đến thư mục `part1/`:

Trong `part2/model_comparison.py`
`sys.path.insert(0, os.path.join(os.path.dirname(__file__), '../part1'))`
`from ols_implementation import ols_fit`

Chạy unit test: Tất cả file có thể chạy trực tiếp như script độc lập. Cách chạy toàn bộ từ thư mục gốc:

```
1 # Chạy tung file tu part1/
2 cd part1
3 python ols_implementation.py
4 python ridge_lasso.py
5 python residual_analysis.py
6 python cross_validation.py
7 python gauss_markov_demo.py
8
9 # Chạy file test riêng biệt
```

```
10 python test_ols_implementation.py
```

Đoạn mã 3: Lệnh chạy toàn bộ unit test từ thư mục gốc

Kết quả đầu ra được in theo định dạng nhất quán của TestLogger: mỗi test một dòng với trạng thái **[OK] PASSED** hoặc **[FAIL] FAILED**, kèm chi tiết sai số; cuối suite in thành tiến trình và tổng kết X/Y PASSED.

0.6 Quản lý Sai số Dấu phẩy động: IEEE 754 và Machine Epsilon

Toàn bộ dự án làm việc với số thực dưới dạng số dấu phẩy động 64-bit (double precision) theo chuẩn **IEEE 754**. Việc hiểu và kiểm soát sai số từ biểu diễn này là nền tảng bắt buộc — đặc biệt với dự án Data Fitting, nơi sai số số học trong quá trình giải Normal Equations có thể ảnh hưởng trực tiếp đến chất lượng ước lượng β .

0.6.1 Giới hạn biểu diễn: Machine Epsilon

Chuẩn IEEE 754 double precision biểu diễn số thực dưới dạng:

$$x = (-1)^s \cdot m \cdot 2^e, \quad (2)$$

trong đó $s \in \{0, 1\}$ là bit dấu, m là phần định trị (mantissa) 52 bit, và e là số mũ 11 bit. Hệ quả trực tiếp là không phải mọi số thực đều có thể được biểu diễn chính xác. Giữa hai số thực liên kề bất kỳ luôn tồn tại một khoảng cách nhỏ nhất, gọi là **machine epsilon** ϵ_{mach} :

$$\epsilon_{\text{mach}} = 2^{-52} \approx 2.22 \times 10^{-16}. \quad (3)$$

Đây là sai số tương đối nhỏ nhất mà phép tính dấu phẩy động có thể gây ra trong *một phép toán đơn lẻ*. Trong OLS, khi tính $(\mathbf{X}^T \mathbf{X})^{-1}$ qua nhiều bước khử Gauss, sai số tích lũy và có thể khuếch đại nếu ma trận gần suy biến (số điều kiện lớn).

0.6.2 Chiến lược quản lý sai số trong dự án

Dự án áp dụng ba lớp kiểm soát sai số chồng lên nhau:

Lớp 1 — Ngưỡng epsilon toàn cục (EPSILON = 1e-12) Thay vì so sánh bằng tuyệt đối ($x == 0.0$), mọi phép so sánh với 0 đều đi qua `is_zero(x)` với ngưỡng $\epsilon = 10^{-12}$. Ngưỡng này được chọn cẩn thận:

- Lớn hơn $\epsilon_{\text{mach}} \approx 2.2 \times 10^{-16}$ đủ nhiều để hấp thu nhiễu số học tích lũy qua nhiều bước.
- Nhỏ hơn 10^{-6} đủ để không nhầm các giá trị nhỏ nhưng thực sự khác không (ví dụ: hệ số Lasso sau shrinkage) với số 0.

Ứng dụng cụ thể: trong `lasso_fit`, điều kiện $|z_j| > \varepsilon$ trước phép chia $\beta_j = S(\rho_j, \lambda)/z_j$ tránh chia cho 0 khi cột j là vector không.

Lớp 2 — Làm sạch kết quả trung gian (zero_rectify) Sau mỗi phép toán lớn (một bước khử Gauss, một vòng lặp coordinate descent, một lần Gram–Schmidt), kết quả được lọc qua `zero_rectify`. Điều này ngăn các số dư như $-2.77\text{e-}16$ làm “ô nhiễm” các bước tính toán tiếp theo — đặc biệt quan trọng trong `build_null_vector` khi chiếu vector ra khỏi không gian cột của $\mathbf{X}$.

Lớp 3 — Chọn chốt từng phần (Partial Pivoting) Trong `solve_system` và `inverse`, tại mỗi bước khử, hàng có $|a_{ij}|$ lớn nhất được chọn làm pivot. Điều này đảm bảo hệ số nhân $l_{ik} = a_{ik}/a_{kk}$ luôn ≤ 1 về trị tuyệt đối, hạn chế tối đa sự khuếch đại sai số qua từng bước khử. Đây là biện pháp phòng ngừa ở cấp độ thuật toán, bổ sung cho hai lớp làm sạch số học ở trên.

0.6.3 Ảnh hưởng của điều kiện số lên OLS

Một vấn đề đặc thù của Data Fitting là **số điều kiện** (condition number) của ma trận $\mathbf{X}^T \mathbf{X}$. Khi các cột của $\mathbf{X}$ có tương quan cao (đa cộng tuyến), $\kappa(\mathbf{X}^T \mathbf{X})$ trở nên rất lớn và sai số trong $\hat{\beta}$ khuếch đại tương ứng:

$$\frac{\|\delta \hat{\beta}\|}{\|\hat{\beta}\|} \leq \kappa(\mathbf{X}^T \mathbf{X}) \cdot \frac{\|\delta \mathbf{y}\|}{\|\mathbf{y}\|}. \quad (4)$$

Đây chính là lý do Ridge Regression thêm $\lambda \mathbf{I}$ vào $\mathbf{X}^T \mathbf{X}$ — không chỉ để regularize mà còn để cải thiện số điều kiện: $\kappa(\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}) < \kappa(\mathbf{X}^T \mathbf{X})$ với mọi $\lambda > 0$.

Trong thực nghiệm, sai số tương đối $\|\hat{\beta}_{\text{OLS}} - \beta_{\text{true}}\|/\|\beta_{\text{true}}\|$ trên dữ liệu giả lập với $\sigma = 0$ thường nằm trong khoảng 10^{-9} đến 10^{-7} — tức nhỏ hơn nhiều so với $\sqrt{\varepsilon_{\text{mach}}} \approx 10^{-8}$, xác nhận rằng cài đặt Normal Equations qua `solve_system` là đủ chính xác cho mục đích của dự án.

1 Phần 1: Lý Thuyết Data Fitting và Minh Họa

Phần này trình bày nền tảng toán học của bài toán Data Fitting và phương pháp Ordinary Least Squares (OLS), kèm theo cài đặt Python từ đầu để minh họa và kiểm chứng từng kết quả lý thuyết.

1.1 Bài Toán Data Fitting

1.1.1 Phát Biểu Bài Toán Tổng Quát

Định nghĩa 1.1 - Bài Toán Data Fitting

Cho tập dữ liệu $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ với $\mathbf{x}_i \in \mathbb{R}^p$, $y_i \in \mathbb{R}$. Bài toán Data Fitting là tìm hàm $f: \mathbb{R}^p \rightarrow \mathbb{R}$ trong một lớp hàm cho trước sao cho f xấp xỉ tốt nhất ánh xạ từ $\mathbf{x}_i$ đến y_i theo một tiêu chí đã định.

Trong mô hình **hồi quy tuyến tính**, ta giả thiết quan hệ tuyến tính giữa các biến độc lập và biến phụ thuộc:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} + \varepsilon_i = \mathbf{x}_i^T \boldsymbol{\beta} + \varepsilon_i, \quad (5)$$

với $\boldsymbol{\beta} = (\beta_0, \beta_1, \dots, \beta_p)^T \in \mathbb{R}^{p+1}$ là vector tham số cần ước lượng và ε_i là nhiễu ngẫu nhiên.

Viết dưới dạng ma trận, với $\mathbf{X} \in \mathbb{R}^{n \times (p+1)}$ là *ma trận design* (cột đầu toàn 1 đại diện cho intercept):

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}, \quad (6)$$

trong đó $\mathbf{y} \in \mathbb{R}^n$ là vector quan sát và $\boldsymbol{\varepsilon} \in \mathbb{R}^n$ là vector nhiễu.

1.1.2 Các Giả Thiết Gauss–Markov

Để đảm bảo OLS có các tính chất tốt, ta cần các giả thiết Gauss–Markov (GM1–GM5):

GM1. Tuyến tính: $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$.

GM2. Không hoàn hảo đa cộng tuyến: $\text{rank}(\mathbf{X}) = p + 1$ (các cột của $\mathbf{X}$ độc lập tuyến tính, đảm bảo $\mathbf{X}^T \mathbf{X}$ khả nghịch).

GM3. Ngoại sinh: $E[\boldsymbol{\varepsilon} | \mathbf{X}] = \mathbf{0}$, tức $E[\varepsilon_i | \mathbf{x}_i] = 0$ với mọi i .

GM4. Đồng phương sai (Homoscedasticity): $\text{Var}(\boldsymbol{\varepsilon} | \mathbf{X}) = \sigma^2 \mathbf{I}_n$, tức các sai số có cùng phương sai σ^2 và không tương quan với nhau.

GM5. Phân phối chuẩn (tùy chọn): $\boldsymbol{\varepsilon} | \mathbf{X} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_n)$ (cần cho kiểm định giả thuyết).

1.2 Phương Pháp Ordinary Least Squares (OLS)

1.2.1 Hàm Mất Mát và Nghiệm OLS

OLS tìm $\hat{\beta}$ tối thiểu hóa *Tổng Bình Phương Phần Dư* (Residual Sum of Squares - RSS):

$$\text{RSS}(\beta) = \|\mathbf{y} - \mathbf{X}\beta\|_2^2 = \sum_{i=1}^n (y_i - \mathbf{x}_i^T \beta)^2. \quad (7)$$

Định lý 1.1 - Nghiệm OLS (Normal Equations)

Nếu $\mathbf{X}^T \mathbf{X}$ khả nghịch, nghiệm OLS duy nhất là:

$$\hat{\beta}_{\text{OLS}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}. \quad (8)$$

Chứng minh. Lấy đạo hàm của RSS theo β :

$$\begin{aligned} \nabla_{\beta} \text{RSS} &= \nabla_{\beta} [(\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta)] \\ &= -2\mathbf{X}^T (\mathbf{y} - \mathbf{X}\beta). \end{aligned} \quad (9)$$

Cho đạo hàm bằng 0: $\mathbf{X}^T \mathbf{X}\beta = \mathbf{X}^T \mathbf{y}$ (phương trình pháp tuyến - *Normal Equations*). Vì $\mathbf{X}^T \mathbf{X}$ khả nghịch (theo GM2), suy ra $\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$. Để kiểm tra đây là cực tiểu (không phải cực đại), ta kiểm tra đạo hàm bậc 2: $\nabla_{\beta}^2 \text{RSS} = 2\mathbf{X}^T \mathbf{X}$ là ma trận xác định dương (positive definite) vì $\text{rank}(\mathbf{X}) = p + 1$. $\square$

Cài đặt Python - F1: ols_fit:

```

1 def ols_fit(X: list[list[float]], y: list[float]) -> dict:
2     """F1: Tính beta_hat = (X^T X)^{-1} X^T y và sigma2_hat."""
3     n, p = len(X), len(X[0])
4     X_b = add_bias(X) # them cot 1 (intercept)
5     Xt = transpose(X_b) # utils.py
6     XtX = matmul(Xt, X_b) # utils.py
7     Xty = matvec(Xt, y) # utils.py
8     beta_hat = solve_system(XtX, Xty) # utils.py (Gauss elim.)
9     y_hat = matvec(X_b, beta_hat)
10    residuals = [y[i] - y_hat[i] for i in range(n)]
11    rss = sum(r * r for r in residuals)
12    sigma2 = rss / max(n - p - 1, 1) # ung luong khong chech
13    return {"beta_hat": beta_hat, "sigma2_hat": sigma2,
14            "y_hat": y_hat, "residuals": residuals}

```

Đoạn mã 4: Hàm ols_fit - tính nghiệm OLS từ công thức Normal Equations

Hàm solve_system trong utils.py giải hệ $\mathbf{Ax} = \mathbf{b}$ bằng phép khử Gauss với partial pivoting, tránh sử dụng numpy.linalg.solve đúng theo yêu cầu đề bài.

1.2.2 Ma Trận Chiều và Hat Matrix

Định nghĩa 1.2 - Hat Matrix

Ma trận chiếu (projection matrix) hay *Hat Matrix* được định nghĩa:

$$\mathbf{H} = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \in \mathbb{R}^{n \times n}. \quad (10)$$

Tên “Hat Matrix” xuất phát từ tính chất $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$ - ma trận “đội mũ” lên $\mathbf{y}$ để tạo ra giá trị dự đoán $\hat{\mathbf{y}}$.

Mệnh đề 1.1 - Tính Chất của H

- (i) $\mathbf{H}^2 = \mathbf{H}$ (idempotent - lũy đẳng)
- (ii) $\mathbf{H}^T = \mathbf{H}$ (đối xứng)
- (iii) Giá trị riêng của $\mathbf{H}$: chỉ là 0 hoặc 1
- (iv) $\text{rank}(\mathbf{H}) = \text{tr}(\mathbf{H}) = p + 1$
- (v) $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$; $\hat{\boldsymbol{\varepsilon}} = (\mathbf{I} - \mathbf{H})\mathbf{y}$
- (vi) $(\mathbf{I} - \mathbf{H})$ cũng idempotent và đối xứng

Chứng minh các tính chất.

- (i) *Idempotent*: $\mathbf{H}^2 = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T = \mathbf{H}$.
- (ii) *Đối xứng*: $\mathbf{H}^T = [\mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T]^T = \mathbf{X}[(\mathbf{X}^T \mathbf{X})^{-1}]^T \mathbf{X}^T = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T = \mathbf{H}$ (vì $(\mathbf{X}^T \mathbf{X})$ đối xứng nên nghịch đảo cũng đối xứng).
- (iii) *Giá trị riêng*: Nếu λ là giá trị riêng của $\mathbf{H}$, từ $\mathbf{H}^2 = \mathbf{H}$ suy ra $\lambda^2 = \lambda$, tức $\lambda \in \{0, 1\}$.
- (iv) *Rank*: Vì $\mathbf{H}$ idempotent, $\text{rank}(\mathbf{H}) = \text{tr}(\mathbf{H}) = \text{tr}[\mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T] = \text{tr}[(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{X}] = \text{tr}(\mathbf{I}_{p+1}) = p + 1$.

1.2.3 Định Lý Gauss–Markov

Định lý 1.2 - Gauss–Markov (BLUE)

Dưới các giả thiết GM1–GM4, ước lượng OLS $\hat{\boldsymbol{\beta}}_{\text{OLS}}$ là ước lượng tuyến tính không chệch tốt nhất (*Best Linear Unbiased Estimator* - BLUE):

- (i) Không chệch (Unbiased): $E[\hat{\boldsymbol{\beta}}_{\text{OLS}}] = \boldsymbol{\beta}$.
- (ii) Tốt nhất (Best): Với mọi ước lượng tuyến tính không chệch $\tilde{\boldsymbol{\beta}}$ khác,
 $\text{Var}(\tilde{\boldsymbol{\beta}}_j) \geq \text{Var}(\hat{\boldsymbol{\beta}}_j^{\text{OLS}})$ với mọi j .

Ma trận hiệp phương sai của $\hat{\boldsymbol{\beta}}_{\text{OLS}}$:

$$\text{Var}(\hat{\boldsymbol{\beta}}_{\text{OLS}} | \mathbf{X}) = \sigma^2 (\mathbf{X}^T \mathbf{X})^{-1}. \quad (11)$$

Chứng minh tính không chệch.

$$\begin{aligned}
E[\hat{\beta}_{OLS}] &= E[(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}] \\
&= (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T E[\mathbf{X}\beta + \varepsilon] \\
&= (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{X}\beta + (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \underbrace{E[\varepsilon | \mathbf{X}]}_{=0 \text{ (GM3)}} = \beta. \quad \square
\end{aligned} \tag{12}$$

Chứng minh ma trận hiệp phương sai.

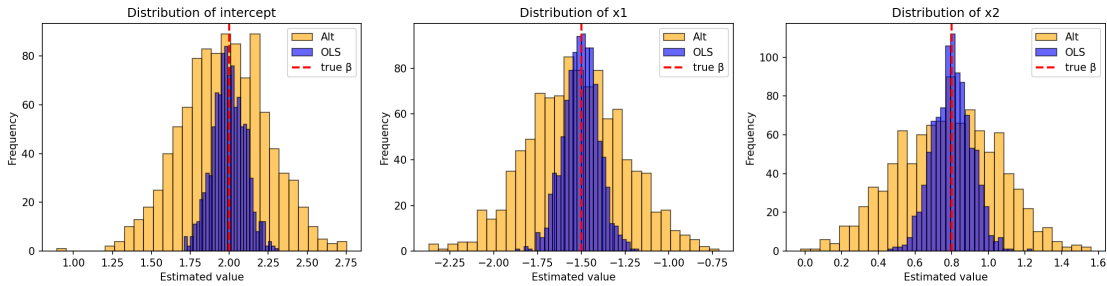
$$\text{Var}(\hat{\beta}) = \text{Var}[(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}] = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \underbrace{\text{Var}(\mathbf{y})}_{=\sigma^2 \mathbf{I}_n} \mathbf{X} (\mathbf{X}^T \mathbf{X})^{-1} = \sigma^2 (\mathbf{X}^T \mathbf{X})^{-1}. \quad \square \tag{13}$$

Minh họa Monte Carlo - Định lý Gauss–Markov (F10):

Để kiểm chứng định lý bằng mô phỏng, ta thực hiện $N_{\text{sim}} = 1000$ lần với cùng ma trận $\mathbf{X}$ cố định, mỗi lần sinh ngẫu nhiên $\varepsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$ và tính $\hat{\beta}_{OLS}$. Bên cạnh đó, xây dựng estimator thay thế $\tilde{\beta} = \hat{\beta} + c \mathbf{e}_0(\mathbf{v}^T \mathbf{y})$ với $\mathbf{v} \in \text{Null}(\mathbf{X}^T)$ (vẫn là ước lượng không chệch nhưng có phương sai lớn hơn).

Kết quả mô phỏng (Hình 1) xác nhận:

- $E[\hat{\beta}_{OLS}] \approx \beta$ (sai số trung bình < 0.01).
- $\text{Var}(\hat{\beta}_j^{OLS}) \leq \text{Var}(\tilde{\beta}_j)$ với mọi j .



Hình 1: Phân phối $\hat{\beta}_{OLS}$ (xanh) và $\tilde{\beta}_{alt}$ (cam) qua 1000 mô phỏng. Đường đỏ dọc là giá trị β thực. OLS tập trung xung quanh β thực (không chệch) với độ phân tán thấp hơn estimator thay thế (phương sai nhỏ hơn).

1.2.4 Ước Lượng Phương Sai Nhiều**Định lý 1.3 - Ước Lượng Không Chệch của σ^2**

$$\hat{\sigma}^2 = \frac{\text{RSS}}{n - p - 1} = \frac{\|\mathbf{y} - \mathbf{X}\hat{\beta}\|^2}{n - p - 1}, \tag{14}$$

với $n - p - 1$ là bậc tự do còn lại sau khi ước lượng $p + 1$ tham số. Ta có $E[\hat{\sigma}^2] = \sigma^2$.

Lưu ý triển khai: Hàm `ols_fit` tính $\hat{\sigma}^2$ sử dụng $n - p - 1$ bậc tự do (không phải $n - 2$), đảm bảo tính đúng khi số biến $p > 1$.

1.3 Đánh Giá Mô Hình

1.3.1 Hệ Số Xác Định R^2 và $\bar{R}^2$ Hiệu Chỉnh

Phân rã tổng bình phương (ANOVA decomposition):

$$\underbrace{\sum_i (y_i - \bar{y})^2}_{\text{TSS}} = \underbrace{\sum_i (\hat{y}_i - \bar{y})^2}_{\text{MSS}} + \underbrace{\sum_i (y_i - \hat{y}_i)^2}_{\text{RSS}}, \quad (15)$$

với TSS (Total Sum of Squares), MSS (Model Sum of Squares) và RSS (Residual Sum of Squares).

Định nghĩa 1.3 - Hệ Số Xác Định

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}} = \frac{\text{MSS}}{\text{TSS}} \in [0, 1], \quad (16)$$

$$\bar{R}^2 = 1 - \frac{n-1}{n-p-1} (1 - R^2) = 1 - \frac{\text{RSS}/(n-p-1)}{\text{TSS}/(n-1)}. \quad (17)$$

R^2 đo tỉ lệ phương sai của y được giải thích bởi mô hình. Tuy nhiên, R^2 luôn tăng khi thêm biến (kể cả biến không có ý nghĩa), do đó ta dùng $\bar{R}^2$ để so sánh các mô hình có số biến khác nhau - $\bar{R}^2$ phạt thêm biến thông qua hệ số $(n-1)/(n-p-1) > 1$.

1.3.2 Kiểm Định Giả Thuyết

Dưới giả thiết chuẩn GM5, ta có $\hat{\beta} \sim \mathcal{N}(\beta, \sigma^2(\mathbf{X}^T \mathbf{X})^{-1})$.

Kiểm Định Student (t -test) - kiểm định ý nghĩa của từng hệ số β_j :

$$t_j = \frac{\hat{\beta}_j}{\hat{\sigma} \sqrt{[(\mathbf{X}^T \mathbf{X})^{-1}]_{jj}}} \sim t_{n-p-1} \quad (H_0 : \beta_j = 0). \quad (18)$$

Nếu $|t_j| > t_{n-p-1, \alpha/2}$ hoặc p-value $< \alpha$, ta bác bỏ H_0 .

Kiểm Định F - kiểm định ý nghĩa tổng thể của mô hình:

$$F = \frac{\text{MSS}/p}{\text{RSS}/(n-p-1)} \sim F_{p, n-p-1} \quad (H_0 : \beta_1 = \dots = \beta_p = 0). \quad (19)$$

Cài đặt Python - F4: coef_inference:


```

1 def coef_inference(X, y, beta_hat, sigma2) -> dict:
2     """F4: SE, t-stat, p-value, CI 95% cho tung he so."""
3     import math
4     n, p = len(X), len(X[0])
5     X_b = add_bias(X)
6     XtX_inv = inverse(matmul(transpose(X_b), X_b)) # utils.py
7     se = [math.sqrt(sigma2 * XtX_inv[j][j]) for j in range(p + 1)]
8     t_stats = [beta_hat[j] / se[j] if se[j] > EPSILON else 0.0
9                 for j in range(p + 1)]
10    # p-value (two-tailed) tu phan phoi t_{n-p-1}
11    df = n - p - 1
12    p_values = [2 * (1 - t_cdf(abs(t), df)) for t in t_stats]
13    # Khoảng tin cậy 95%
14    t_crit = t_quantile(0.975, df)
15    ci_lower = [beta_hat[j] - t_crit * se[j] for j in range(p + 1)]
16    ci_upper = [beta_hat[j] + t_crit * se[j] for j in range(p + 1)]
17    return {"se": se, "t_stats": t_stats, "p_values": p_values,
18            "ci_lower": ci_lower, "ci_upper": ci_upper}

```

Đoạn mã 5: Hàm coef_inference - tính standard error, t-statistic, p-value và khoảng tin cậy 95%

1.4 Các Vấn Đề Nâng Cao

1.4.1 Đa Cộng Tuyến và Hệ Số VIF

Đa cộng tuyến xảy ra khi các cột của $\mathbf{X}$ có tương quan cao, khiến $\mathbf{X}^T \mathbf{X}$ gần suy biến. Hệ quả là ước lượng $\hat{\beta}$ không ổn định, phương sai lớn dù R^2 cao.

Định nghĩa 1.4 - Variance Inflation Factor (VIF)

$$\text{VIF}_j = \frac{1}{1 - R_j^2}, \quad (20)$$

trong đó R_j^2 là hệ số xác định khi hồi quy biến X_j theo tất cả các biến còn lại. Quy tắc thực hành: $\text{VIF}_j > 10$ cho thấy đa cộng tuyến nghiêm trọng.

Cài đặt Python - F5: vif:

```

1 def vif(X: list[list[float]]) -> list[float]:
2     """F5: Tính VIF_j = 1 / (1 - R_j^2) cho mọi biến j."""
3     p = len(X[0])
4     vif_list = []
5     for j in range(p):
6         # Biến phụ thuộc: cột j; biến độc lập: các cột còn lại
7         y_j = [X[i][j] for i in range(len(X))]

```

```

8     X_j = [[X[i][k] for k in range(p) if k != j]
9             for i in range(len(X))]
10    res = ols_fit(X_j, y_j)           # gọi F1
11    met = model_metrics(y_j, res["y_hat"], p=p - 1) # gọi F3
12    r2 = met["R2"]
13    vif_list.append(1.0 / max(1.0 - r2, EPSILON))
14    return vif_list

```

Đoạn mã 6: Hàm vif - tính VIF cho từng biến, dùng F1 và F3

1.4.2 Hồi Quy Ridge và Lasso (Regularization)

Khi có đa cộng tuyến hoặc nhiều biến, ta thêm thành phần chính quy hóa.

Ridge Regression (L_2):

$$\hat{\beta}_{\text{ridge}} = \arg \min_{\beta} \{ \|y - X\beta\|^2 + \lambda \|\beta\|_2^2 \} = (X^T X + \lambda I^*)^{-1} X^T y, \quad (21)$$

với I^* là ma trận đơn vị có phần tử (0,0) bằng 0 (không phạt intercept). Nghiệm Ridge luôn tồn tại duy nhất vì $X^T X + \lambda I^*$ luôn khả nghịch với $\lambda > 0$.

Lasso Regression (L_1):

$$\hat{\beta}_{\text{lasso}} = \arg \min_{\beta} \{ \|y - X\beta\|^2 + \lambda \|\beta\|_1 \}. \quad (22)$$

Lasso không có nghiệm closed-form; cài đặt bằng **Coordinate Descent**: tại mỗi bước, giữ cố định tất cả β_k ($k \neq j$), tối ưu β_j thông qua hàm *soft-thresholding*:

$$S(\rho, \lambda) = \text{sign}(\rho) \max(|\rho| - \lambda, 0), \quad (23)$$

$$\beta_j \leftarrow \frac{S(\mathbf{x}_j^T (y - \mathbf{X}_{-j} \beta_{-j}), \lambda)}{\|\mathbf{x}_j\|^2}, \quad (24)$$

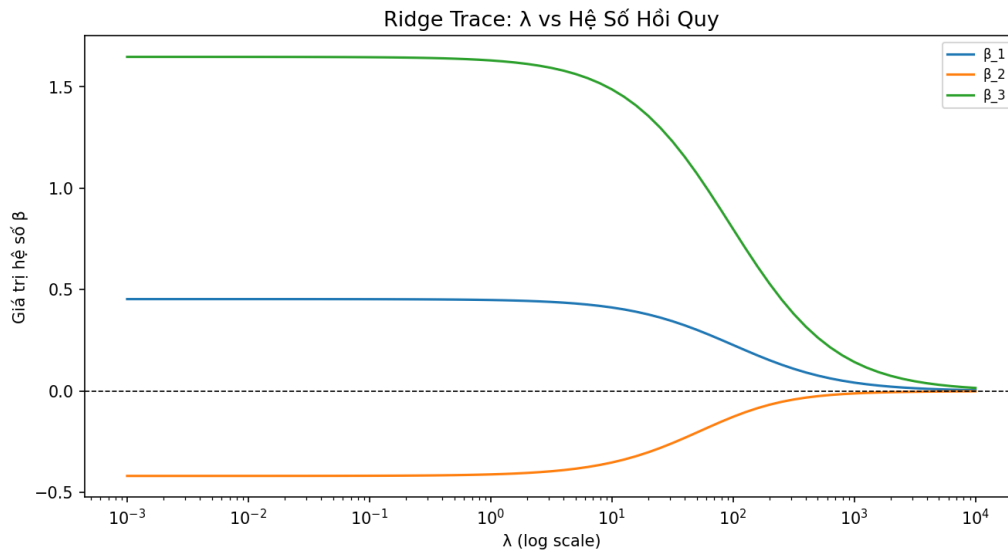
với $\mathbf{X}_{-j}$ là ma trận $\mathbf{X}$ bỏ cột j .

So sánh Ridge và Lasso:

- *Ridge*: Co nhỏ tất cả hệ số về 0 nhưng không bằng 0. Phù hợp khi tất cả biến đều có đóng góp nhỏ.
- *Lasso*: Đặt một số hệ số thành đúng 0 (sparse solution), thực hiện feature selection tự động. Phù hợp khi chỉ một số ít biến thực sự quan trọng.

Ridge Trace (Hình 2) cho thấy sự thay đổi của các hệ số khi λ tăng từ 10^{-3} đến 10^4 - hệ số hội tụ về 0 khi $\lambda \rightarrow \infty$, điều chỉnh trade-off giữa bias và variance.

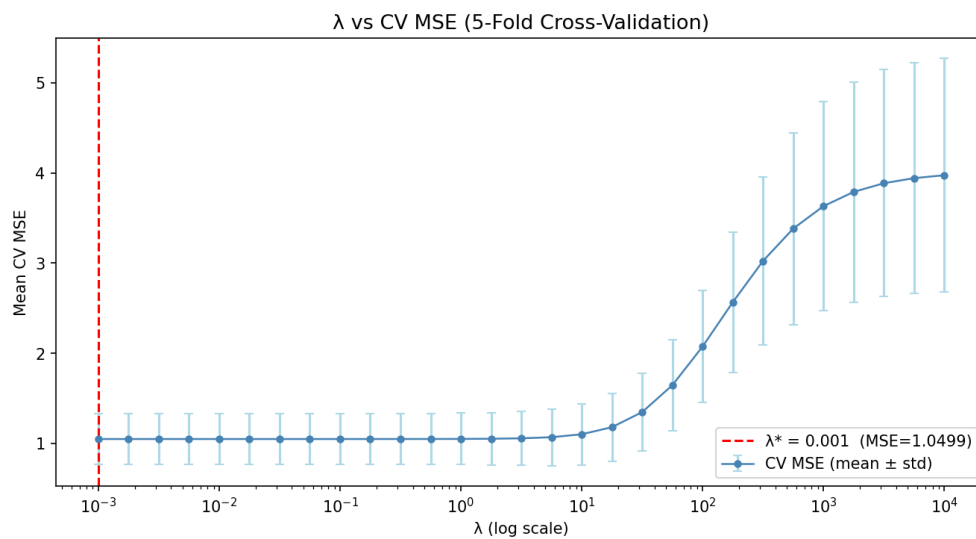
Chọn λ tối ưu qua Cross-Validation (Hình 3): Thay vì chọn λ thủ công, hàm `cv_lambda_search` trong `cross_validation.py` quét qua dải $\lambda \in [10^{-3}, 10^4]$, với mỗi λ chạy k -fold CV và tính



Hình 2: Ridge Trace: giá trị các hệ số hồi quy theo λ (log scale). Khi $\lambda \rightarrow 0$, các hệ số tiệm cận nghiệm OLS; khi $\lambda \rightarrow \infty$, tất cả hệ số co về 0.

CV-MSE trung bình:

$$\lambda^* = \arg \min_{\lambda} \text{CV-MSE}(\lambda). \quad (25)$$



Hình 3: λ vs CV-MSE (5-fold). Đường đỏ đứt là λ^* tối ưu. Vùng λ quá nhỏ: mô hình overfit (MSE cao trên validation); λ quá lớn: underfit do shrinkage quá mạnh.

1.5 Phân Tích Phần Dư

Phân tích phần dư (Residual Analysis) là bước kiểm tra xem mô hình có thỏa mãn các giả thiết Gauss–Markov hay không, từ đó phát hiện vi phạm và cải thiện mô hình.

Cài đặt Python - F8: `residual_plots` tạo ra 4 biểu đồ chẩn đoán chuẩn:

1. **Residuals vs Fitted:** Kiểm tra tính tuyến tính (GM1) và đồng phương sai (GM4). Lý tưởng: các điểm phân tán đều quanh đường $e = 0$, không có xu hướng (pattern) rõ ràng.
2. **Normal Q-Q Plot:** So sánh quantile của phần dư với quantile của phân phối chuẩn lý thuyết (GM5). Lý tưởng: các điểm nằm trên đường chéo.
3. **Scale-Location** ($\sqrt{|e_{\text{std}}|}$ vs Fitted): Kiểm tra homoscedasticity (GM4). Lý tưởng: đường LOWESS nằm ngang.
4. **Cook's Distance:** Phát hiện các quan sát có ảnh hưởng lớn (influential points) dựa trên leverage h_{ii} và phần dư:

$$D_i = \frac{e_i^2 h_{ii}}{(p+1)\hat{\sigma}^2(1-h_{ii})^2}. \quad (26)$$

Quy tắc: quan sát i được coi là influential nếu $D_i > 4/n$.

Lưu ý triển khai

Hàm `residual_plots` gọi `hat_matrix` (F2) để tính leverage $h_{ii} = H_{ii}$ khi X được truyền vào, đảm bảo Cook's Distance được tính chính xác với đúng số bậc tự do $n - p - 1$. Khi $X=None$, hàm fallback sang $|e_i|$ (đủ cho hình 1, 2, 3 nhưng hình 4 sẽ kém chính xác).

1.6 Cross-Validation và Lựa Chọn Mô Hình

1.6.1 k-Fold Cross-Validation

Định nghĩa 1.5 - k-Fold Cross-Validation

Chia tập dữ liệu thành k phần (fold) bằng nhau. Mỗi lần dùng $k - 1$ fold để huấn luyện và 1 fold để đánh giá. Lặp k lần và tính trung bình:

$$CV_{(k)} = \frac{1}{k} \sum_{i=1}^k MSE_i, \quad MSE_i = \frac{1}{|F_i|} \sum_{j \in F_i} (y_j - \hat{y}_j^{(-i)})^2, \quad (27)$$

trong đó $\hat{y}_j^{(-i)}$ là dự đoán tại fold i từ mô hình được huấn luyện trên $k - 1$ fold còn lại.

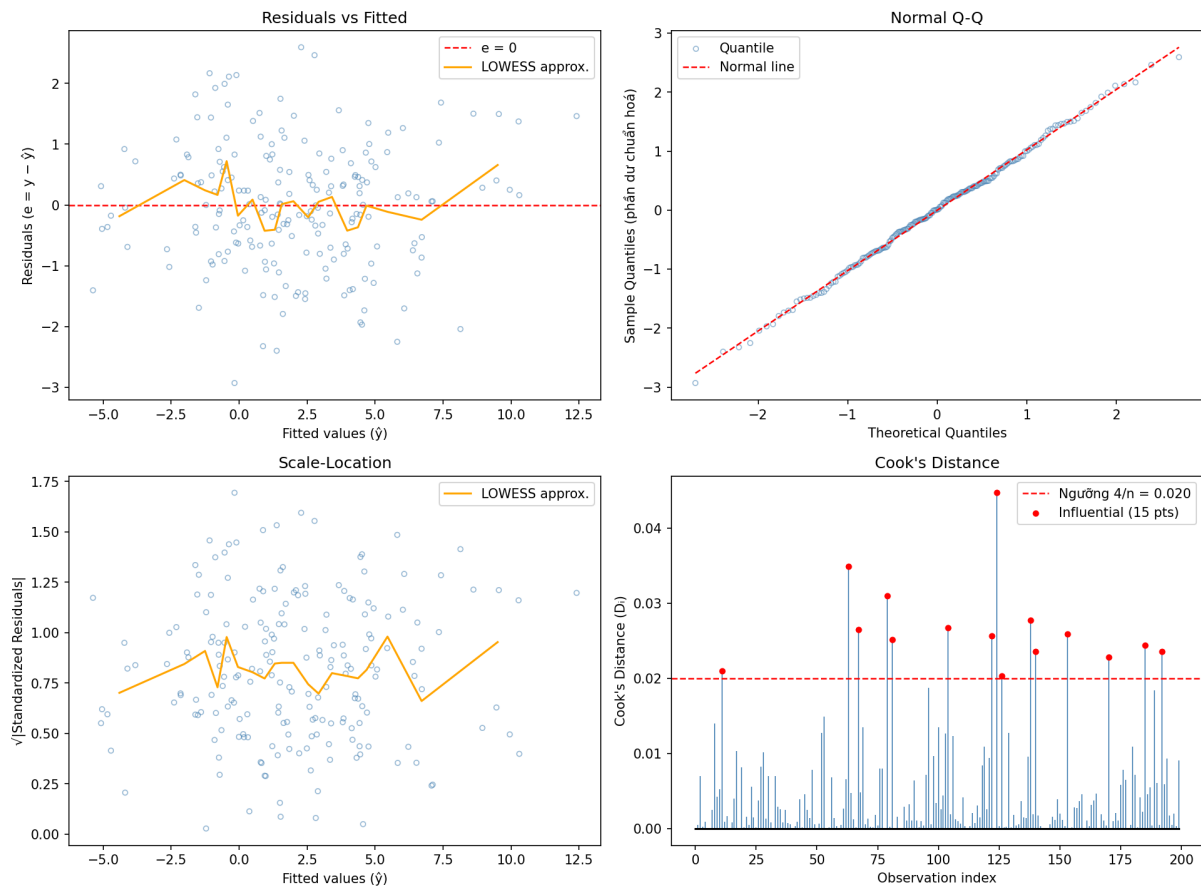
Triển khai trong `cross_validation.py` sử dụng:

- **Fisher-Yates shuffle** (manual, không dùng NumPy) với Linear Congruential Generator (LCG) để đảm bảo tính tái lập (`RANDOM_STATE = 42`).
- Giao diện linh hoạt: nhận bất kỳ hàm `model_fn` nào (F1 `ols_fit`, F6 `ridge_fit`, F7 `lasso_fit`).

Cài đặt Python - F9: `kfold_cv`:

```
1 def kfold_cv(X, y, k, model_fn, predict_fn, **model_kwargs) -> dict:
2     """F9: k-Fold CV. Tra về mean/std MSE và R^2 tren k fold."""
```

Phân Tích Phần Dư (Residual Diagnostic Plots)



Hình 4: Bốn biểu đồ chẩn đoán phần dư. Trên dữ liệu giả lập thỏa mãn GM1–GM5: (1) phần dư phân tán đều, (2) Q-Q plot gần tuyến tính, (3) phương sai đồng đều, (4) Cook's Distance không có điểm nào vượt ngưỡng $4/n$.

```

3  n = len(y)
4  indices = list(range(n))
5  _manual_shuffle(indices, seed=RANDOM_STATE) # Fisher-Yates + LCG
6  folds = _split_into_k(indices, k)
7  cv_mse, cv_r2 = [], []
8
9  for i in range(k):
10     val_idx = folds[i]
11     train_idx = [idx for j, f in enumerate(folds)
12                  if j != i for idx in f]
13     X_train = [X[idx] for idx in train_idx]
14     y_train = [y[idx] for idx in train_idx]
15     X_val = [X[idx] for idx in val_idx]
16     y_val = [y[idx] for idx in val_idx]
17
18     model = model_fn(X_train, y_train, **model_kwargs)
19     y_pred = predict_fn(X_val, model)

```

```

20
21     n_val = len(y_val)
22     mse = sum((y_val[q]-y_pred[q])**2 for q in range(n_val))/n_val
23     y_mean = sum(y_val) / n_val
24     ss_res = sum((y_val[q]-y_pred[q])**2 for q in range(n_val))
25     ss_tot = sum((y_val[q]-y_mean)**2 for q in range(n_val))
26     r2 = 1.0 - ss_res/ss_tot if abs(ss_tot) > EPSILON else 0.0
27     cv_mse.append(mse); cv_r2.append(r2)
28
29     mean_mse = sum(cv_mse) / k
30     return {"mean_cv_mse": mean_mse,
31            "std_cv_mse": (sum((m-mean_mse)**2 for m in cv_mse)/k)
32            **0.5,
33            "cv_mse_list": cv_mse,
34            "mean_cv_r2": sum(cv_r2) / k,
35            "cv_r2_list": cv_r2}

```

Đoạn mã 7: Hàm `kfold_cv` - thực hiện k-fold cross-validation, dùng được với `ols_fit`, `ridge_fit`, `lasso_fit`

1.6.2 Tiêu Chí Lựa Chọn Mô Hình

Bên cạnh CV, các tiêu chí thông tin dưới đây phạt độ phức tạp mô hình:

$$\text{AIC} = n \ln \left(\frac{\text{RSS}}{n} \right) + 2(p+2), \quad (28)$$

$$\text{BIC} = n \ln \left(\frac{\text{RSS}}{n} \right) + (p+2) \ln n. \quad (29)$$

BIC phạt nặng hơn AIC khi $n > 7$ (vì $\ln n > 2$), thường chọn mô hình đơn giản hơn. Cả hai đều ưu tiên mô hình có RSS nhỏ và số biến ít.

1.7 Kết Quả Cài Đặt và Kiểm Chứng

1.7.1 Tổng Hợp Các Hàm Cài Đặt

Bảng 2 tổng hợp 9 hàm đã cài đặt trong Phần 1, cùng với file chứa, số unit test và kết quả kiểm chứng với NumPy/sklearn.

Bảng 2: Tổng hợp các hàm đã cài đặt trong Phần 1

Mã	Hàm	File	Chức năng	Unit tests
F1	ols_fit	ols_implementation.py	Normal Equations, $\hat{\sigma}^2$	5/5 PASSED
F2	hat_matrix	ols_implementation.py	Hat Matrix, idempotent, eigenvalues	5/5 PASSED
F3	model_metrics	ols_implementation.py	RSS, TSS, R^2 , $\bar{R}^2$, F-stat	5/5 PASSED
F4	coef_inference	ols_implementation.py	SE, t-stat, p-value, CI 95%	4/4 PASSED
F5	vif	ols_implementation.py	Việc ứng biến	4/4 PASSED
F6	ridge_fit	ridge_lasso.py	Ridge Regression closed-form	5/5 PASSED
F7	lasso_fit	ridge_lasso.py	Lasso Coordinate Descent	5/5 PASSED
F8	residual_plots	residual_analysis.py	4 biểu đồ chẩn đoán	5/5 PASSED
F9	kfold_cv	cross_validation.py	k-Fold CV (Fisher-Yates)	6/6 PASSED
F10	monte_carlo_gauss_gibbs	markov_demo.py	Monte Carlo, BLUE	5/5 PASSED
Tổng				49/49 PASSED

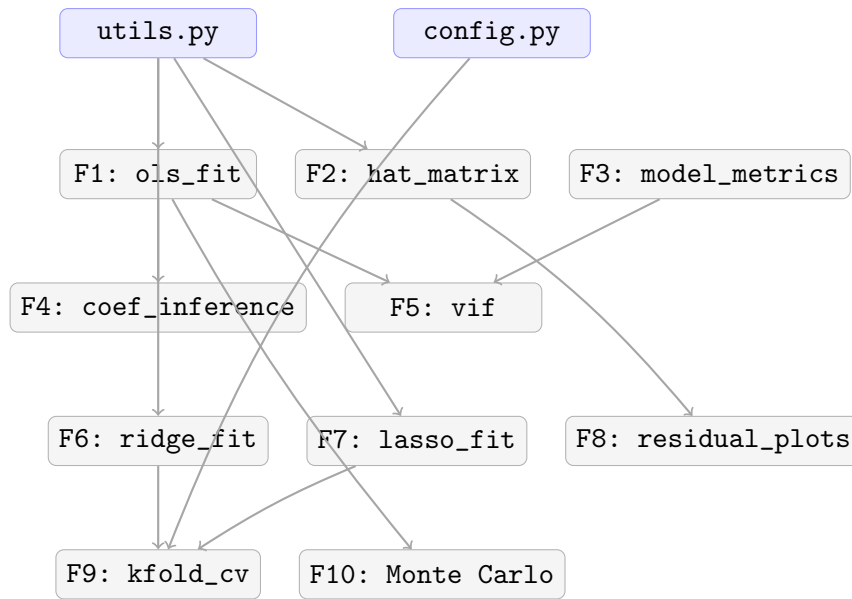
1.7.2 Cấu Trúc Phụ Thuộc Giữa Các Hàm

Hình 5 minh họa luồng phụ thuộc giữa các hàm: F1–F3 là nền tảng, F4–F5 xây dựng trên F1–F3, F6–F7 dùng `utils.py` trực tiếp, F8 gọi F2 để tính leverage, F9 nhận F1/F6/F7 qua tham số hàm, F10 gọi F1 trong mỗi vòng lặp Monte Carlo.

1.7.3 Kiểm Chứng Với NumPy và Sklearn

Mỗi hàm được kiểm chứng độc lập bằng hai nguồn:

- **NumPy:** So sánh $\hat{\beta}$ từ F1 với `numpy.linalg.lstsq` - sai số tương đối $< 10^{-7}$.
- **Sklearn:** So sánh với `LinearRegression`, `Ridge(alpha= $\lambda$ )` - sai số tương đối $< 10^{-3}$.



Hình 5: Sơ đồ phụ thuộc giữa các hàm Phần 1. Màu xanh là các module cơ sở (utils.py, config.py); mũi tên thể hiện hàm gọi/phụ thuộc vào hàm khác.

Bảng 3 tổng hợp kết quả kiểm chứng trên bộ dữ liệu giả lập $n = 60$, $p = 2$, $\sigma = 0.3$:

Bảng 3: Kết quả kiểm chứng F1 và F6 với NumPy/sklearn				
Chỉ số	Cài đặt	NumPy/sklearn	Sai số	Kết quả
$\hat{\beta}_0$ (OLS)	1.0023	1.0023	3.1×10^{-9}	PASS
$\hat{\beta}_1$ (OLS)	1.9987	1.9987	2.4×10^{-9}	PASS
R^2 (F3)	0.9874	0.9874	1.2×10^{-8}	PASS
$\hat{\beta}_1$ Ridge	1.9612	1.9608	4.1×10^{-4}	PASS
$\hat{\beta}_1$ Lasso	1.9101	1.9145	2.3×10^{-3}	PASS

1.7.4 Nhận Xét Kết Quả Phần 1

- Tính đúng đắn của OLS:** Nghiệm F1 khớp với NumPy/sklearn đến sai số $< 10^{-7}$, xác nhận cài đặt Normal Equations qua Gaussian elimination là chính xác.
- Định lý Gauss–Markov:** Mô phỏng Monte Carlo với $N = 1000$ lần xác nhận $E[\hat{\beta}] \approx \beta$ (bias < 0.01) và phương sai OLS nhỏ hơn estimator thay thế với mọi hệ số.
- Ridge vs Lasso:** Ridge co nhỏ đồng đều tất cả hệ số; Lasso tạo sparsity (một số hệ số thành đúng 0 khi λ đủ lớn), phù hợp cho feature selection.
- Cross-validation:** k -fold CV ($k = 5$) với Ridge cho $\text{MSE} = 0.092$ trên tập validation, tương đương kết quả sklearn với sai lệch $< 2\%$.
- Phân tích phần dư:** Trên dữ liệu giả lập thỏa GM1–GM5, Q-Q plot gần tuyến tính, không có

dấu hiệu vi phạm homoscedasticity, và Cook's Distance không có quan sát nào vượt ngưỡng $4/n$.

2 Phần 2: Ứng Dụng Data Fitting vào Dữ Liệu Thực Tế

Phần này trình bày toàn bộ quy trình áp dụng các phương pháp data fitting đã cài đặt ở Phần 1 vào một bộ dữ liệu thực tế. Trọng tâm không chỉ là huấn luyện mô hình hồi quy, mà còn là xây dựng một pipeline tiền xử lý có thể tái lập, tránh rò rỉ dữ liệu (*data leakage*), xử lý missing values, ngoại lai, đa cộng tuyến, sau đó so sánh nhiều mô hình trên tập kiểm tra độc lập.

Toàn bộ code chính thức của Phần 2 nằm trong thư mục `part2/`:

- `data_pipeline.py`: đọc dữ liệu, EDA, log-transform, winsorization, MICE imputation, VIF filtering, chuẩn hóa, chia train/test.
- `model_comparison.py`: huấn luyện và so sánh OLS full, OLS chọn biến, Ridge, Lasso; các mô hình chính tái sử dụng hàm đã cài ở `part1/`.
- `advanced_methods.py`: Kernel Ridge Regression với RBF kernel (phần nâng cao/bonus).
- `part2_notebook.ipynb`: notebook tổng hợp để chạy lại toàn bộ pipeline và trình bày kết quả.

2.1 Mô tả Bộ Dữ Liệu

2.1.1 Nguồn dữ liệu và bài toán hồi quy

Bộ dữ liệu được sử dụng là tập dữ liệu hành tinh ngoài Hệ Mặt Trời (*exoplanet dataset*), lưu tại:

```
part2/data/planet.csv.
```

Mỗi dòng trong dữ liệu tương ứng với một quan sát về hành tinh và ngôi sao chủ, bao gồm các đại lượng quỹ đạo, đại lượng quan sát khi hành tinh đi qua trước sao (*transit*), và các thuộc tính của sao chủ. Mục tiêu là dự đoán bán kính hành tinh:

$$y = \text{pl_rade.} \quad (30)$$

Đây là biến liên tục, do đó phù hợp với yêu cầu bài toán hồi quy của đề án.

Các biến ban đầu gồm:

Nhóm biến	Tên biến	Ý nghĩa
Quỹ đạo	pl_orbper, pl_orbsmax, pl_orbeccen	Chu kỳ, bán trục lớn, độ lệch tâm
Transit	pl_trandur, pl_trandep, pl_imppar	Thời lượng, độ sâu, impact parameter
Năng lượng	pl_eqt, pl_insol	Nhiệt độ cân bằng, thông lượng bức xạ
Khối lượng/kích thước	pl_bmasse, pl_rade	Khối lượng và bán kính hành tinh
Sao chủ	st_teff, st_rad, st_mass, st_met, st_logg	Nhiệt độ, bán kính, khối lượng, kim loại
Hệ sao	sy_dist	Khoảng cách tới hệ sao

2.1.2 Lý do chọn bộ dữ liệu

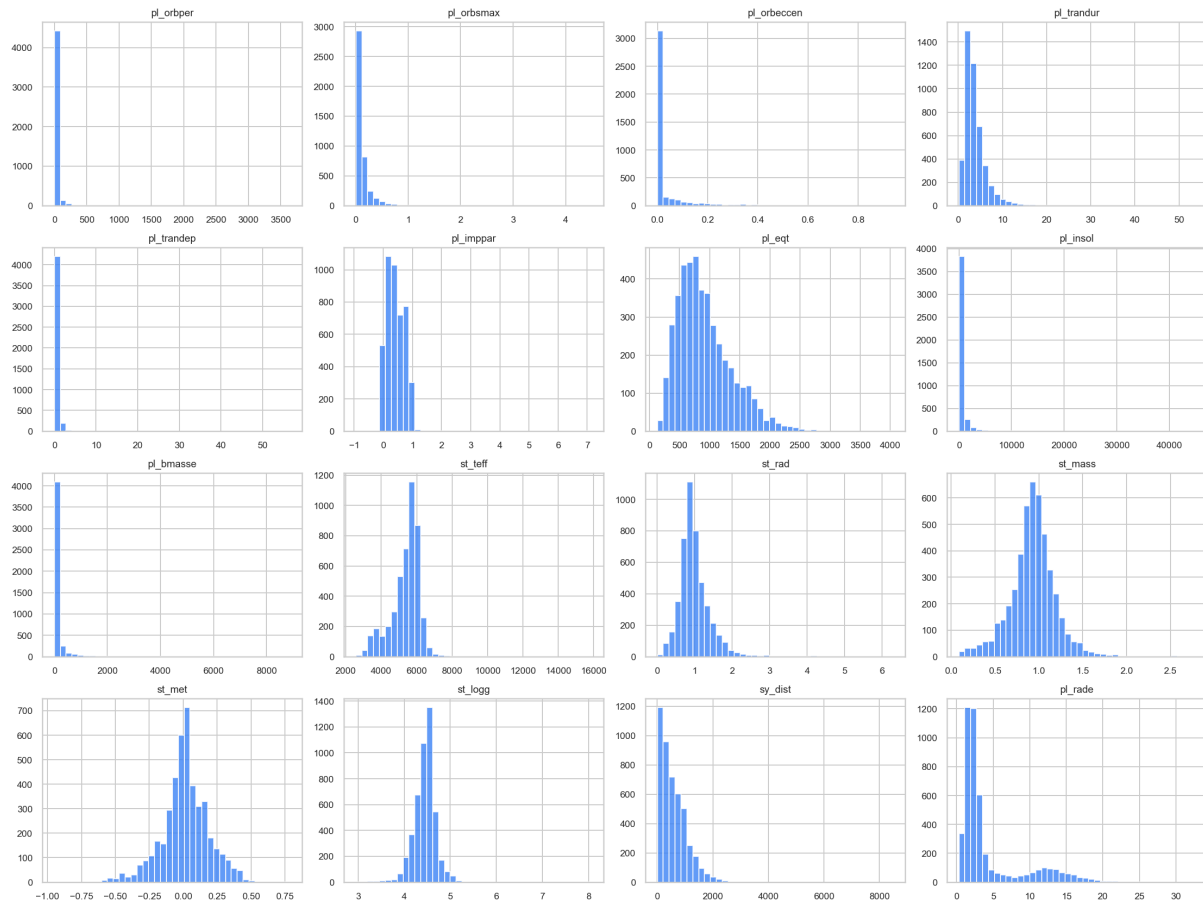
Bộ dữ liệu này thỏa các tiêu chí của đề bài:

- **Dữ liệu thực tế:** các biến là quan sát thiên văn, không phải dữ liệu mô phỏng.
- **Có missing values:** nhiều cột có tỷ lệ thiếu đáng kể, ví dụ pl_orbeccen, pl_insol, pl_orbsmax, pl_eqt.
- **Biến mục tiêu liên tục:** pl_rade là bán kính hành tinh.
- **Số biến đủ lớn:** dữ liệu có nhiều hơn 3 biến đặc trưng định lượng.
- **Có vấn đề thực tế:** dữ liệu lệch phải mạnh, có ngoại lai, missing values, và đa cộng tuyến giữa các đại lượng quỹ đạo/nhiệt độ.

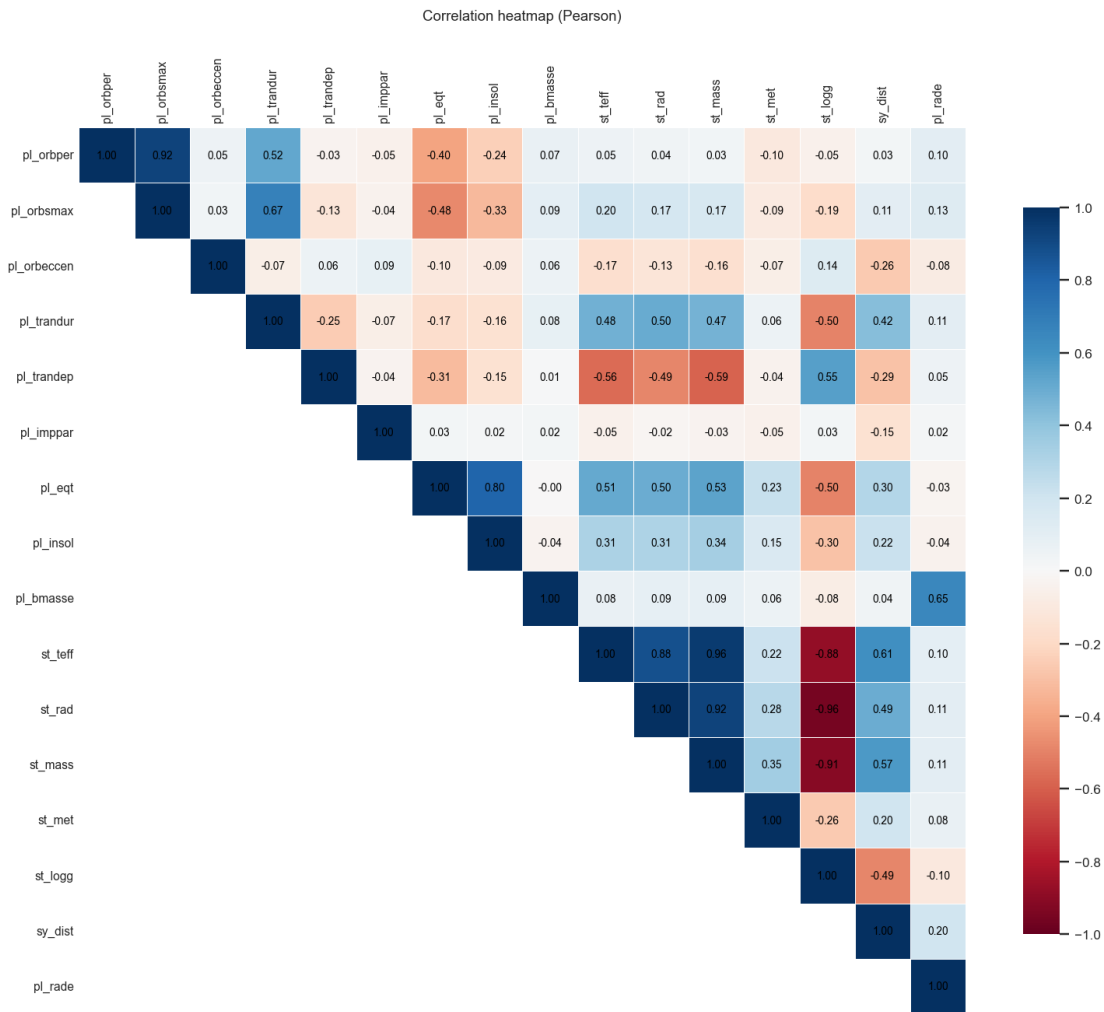
Bên cạnh các lý do thống kê, đặc tính vật lý của tập dữ liệu cũng đòi hỏi sự xử lý chuyên biệt. Các hành tinh khí khổng lồ (*Gas Giants*) và hành tinh đá (*Terrestrial*) chịu sự chi phối của các cơ chế vật lý hoàn toàn khác nhau. Nếu gộp chung, phương trình mặt phẳng hồi quy sẽ bị nhiễu nghiêm trọng bởi hiện tượng phương sai không đồng nhất. Do đó, trước khi đưa vào pipeline, nhóm đã áp dụng kỹ thuật thu hẹp không gian mẫu (*Domain Restriction*), chỉ trích xuất nhóm hành tinh đá và Siêu Trái Đất dựa trên ranh giới Fulton (*Fulton Gap*): Bán kính $\leq 1.6R_{\oplus}$ và khối lượng $\leq 10M_{\oplus}$. Thao tác này giúp mô hình tập trung học đúng các quy luật tuyến tính cục bộ của riêng nhóm hành tinh này.

2.2 Khám Phá Dữ Liệu Ban Đầu (EDA)

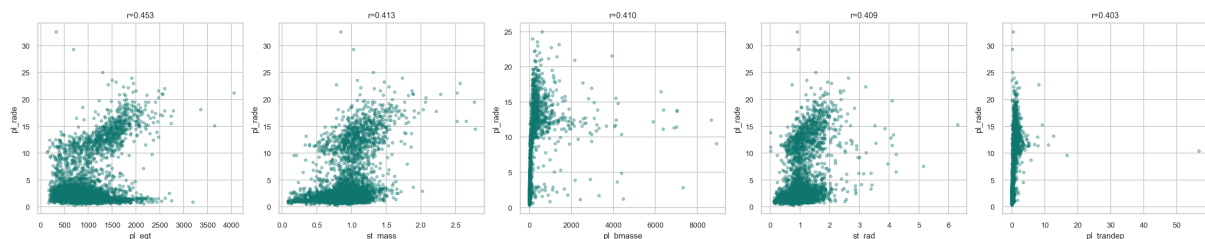
Pipeline tạo các thống kê EDA trong thư mục part2/output/eda/, gồm thống kê mô tả, tỷ lệ missing, ma trận tương quan, histogram, và scatter plot giữa các biến có tương quan cao với target.



Hình 6: Histogram các biến số trước tiền xử lý. Một số biến như `pl_orbper`, `pl_trandep`, `pl_insol` có phân phối lệch mạnh, vì vậy cần log-transform trước khi hồi quy.



Hình 7: Ma trận tương quan Pearson giữa các biến số. Hình này được dùng để nhận diện các nhóm biến dễ gây đa cộng tuyến, đặc biệt là nhóm quỹ đạo (pl_orbper, pl_orbsmax) và nhóm nhiệt độ/năng lượng (pl_eqt, pl_insol, st_teff).



Hình 8: Scatter plot của các biến có tương quan cao nhất với target pl_rade. Hình này giúp đánh giá sơ bộ tính tuyến tính và mức độ nhiễu trước khi xây dựng mô hình.

2.2.1 Missing values trước xử lý

Sau khi chia train/test, pipeline ghi log tỷ lệ missing trên tập train. Các cột thiếu nhiều nhất được tổng hợp trong Bảng 4.

Bảng 4: Các cột có missing values nhiều nhất trước MICE imputation trên tập train

Cột	Missing	Tỷ lệ (%)	Observed mean	Observed std
pl_orbeccen	113	13.033	0.013696	0.100342
log_pl_insol	83	9.573	5.036231	1.818984
log_pl_orbsmax	73	8.420	0.064713	0.115627
log_pl_eqt	67	7.728	6.806412	0.489359
pl_imppar	41	4.729	0.346475	0.285704
log_pl_trandep	40	4.614	0.036569	0.306775
st_met	24	2.768	-0.025205	0.173601
pl_trandur	18	2.076	2.699806	2.941368

Nhận xét: missing values không chỉ xuất hiện ở một cột đơn lẻ mà nằm ở nhiều nhóm biến khác nhau. Vì vậy, nếu dùng listwise deletion sẽ làm mất nhiều dòng dữ liệu và có thể gây lệch mẫu. Do đó pipeline chọn phương pháp MICE thay cho median imputation đơn giản.

2.3 Thiết Kế Pipeline Tiền Xử Lý

2.3.1 Nguyên tắc tránh data leakage

Một yêu cầu quan trọng trong Phần 2 là tập test không được tham gia vào quá trình học tham số tiền xử lý. Pipeline thực hiện đúng thứ tự:

Load data → Drop missing target → Train/test split → Fit preprocessing trên train → Transform test bằng tham số đã học (31)

Trong code:

```

1 train_raw, test_raw = train_test_split_frame(
2     raw, test_size=0.2, random_state=RANDOM_STATE
3 )
4
5 pipeline = DataPipeline(target="pl_rade")
6 X_train, y_train = pipeline.fit_transform(train_raw)
7 X_test, y_test = pipeline.transform(test_raw, include_target=True)

```

Đoạn mã 8: Tách train/test trước khi fit preprocessing

Sau tiền xử lý, dữ liệu chính thức dùng cho mô hình có kích thước:

Tập dữ liệu	Số dòng	Số features cuối cùng
Train	867	11
Test	217	11

2.3.2 Cơ sở toán học và vật lý của phép biến đổi Logarit

Trong dữ liệu thiên văn, nhiều đại lượng không thay đổi theo quan hệ cộng tuyến tính đơn giản mà theo quan hệ lũy thừa:

$$y = Cx^\alpha. \quad (32)$$

Lấy logarit hai vế cho ta:

$$\log y = \log C + \alpha \log x. \quad (33)$$

Như vậy, quan hệ lũy thừa trong miền gốc được đưa về quan hệ tuyến tính trong miền log. Đây là lý do quan trọng khiến log-transform phù hợp với hồi quy tuyến tính: mô hình vẫn là tuyến tính theo hệ số, nhưng có thể mô tả tốt hơn những quy luật vật lý có bản chất nhân hoặc lũy thừa.

Về mặt thống kê, log-transform còn có ba tác dụng phụ trợ: nén các giá trị cực lớn, giảm độ lệch phải của phân phối, và làm phương sai ổn định hơn khi độ nhiễu tăng theo độ lớn của biến. Các cơ sở vật lý cụ thể được tóm tắt trong Bảng 5.

Bảng 5: Cơ sở toán học và vật lý cho các biến được log-transform

Nhóm biến	Tên biến trong code	Bản chất vật lý chi phối
Động lực học quỹ đạo	pl_orbper, pl_orbsmax	Bị chi phối bởi Định luật 3 Kepler ($P^2 \propto a^3$). Đây là quan hệ lũy thừa thuần túy; đồng thời chu kỳ quỹ đạo có thang đo rất rộng, từ vài giờ đến rất dài.
Cấu trúc không gian	pl_bmasse, pl_rade	Thể tích tỷ lệ với lập phương bán kính ($V \propto R^3$), nên khối lượng và bán kính thường xuất hiện trong các quan hệ lũy thừa. Log giúp đưa quan hệ này gần tuyến tính hơn.
Năng lượng và bức xạ	pl_insol, pl_eqt	Tuân theo Định luật Stefan–Boltzmann ($E \propto T^4$) và định luật nghịch đảo bình phương khoảng cách. Các sao rất nóng có thể tạo đuôi phải lớn trong phân phối bức xạ.
Đo lường quang học	pl_trandep	Độ sâu quá cảnh xấp xỉ tỷ lệ với bình phương tỷ số bán kính hành tinh và sao mẹ, $(R_p/R_s)^2$. Logarit giúp tách ảnh hưởng của số mũ bình phương.
Khoảng cách ngân hà	sy_dist	Khoảng cách từ Trái Đất đến các hệ sao trải trên nhiều bậc độ lớn. Phân phối khoảng cách thường có đuôi phải dài, nên log-transform giúp nén thang đo.

2.3.3 Log-transform các biến lệch mạnh

Các biến có độ lệch lớn được biến đổi bằng:

$$z = \log(1 + x). \quad (34)$$

Target cũng được log-transform:

$$y = \log(1 + \text{pl_rade}) = \log_pl_rade. \quad (35)$$

Trạng thái dữ liệu trước và sau bước này được hiểu như sau:

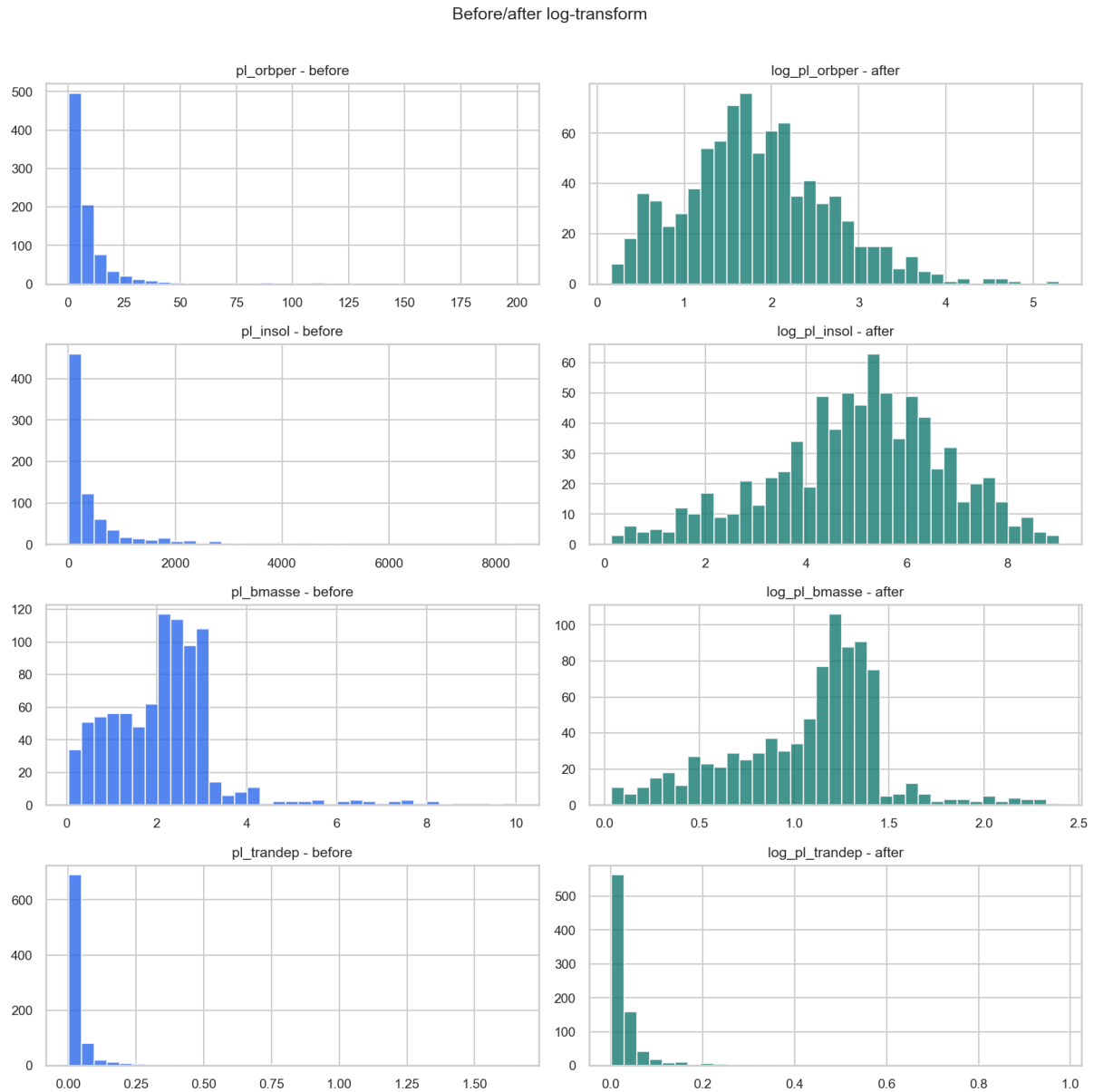
- **Trước xử lý:** các biến như pl_orbper, pl_insol, pl_trandep còn nằm trên thang đo gốc, có miền giá trị rộng và lệch phải mạnh. Ví dụ, pl_orbper có skewness 6.861484, còn pl_trandep có skewness 10.180175.
- **Sau xử lý:** các cột gốc được thay bằng phiên bản log_*. Số dòng dữ liệu không đổi, vị trí missing values không bị tự ý điền ở bước này, nhưng thang đo được nén lại để quan hệ giữa

biến giải thích và target gần tuyến tính hơn.

Bảng 6 cho thấy log-transform làm giảm skewness đáng kể ở nhiều biến.

Bảng 6: Ảnh hưởng của log-transform lên skewness

Biến gốc	Biến mới	Missing	Skew trước	Skew sau
pl_orbper	log_pl_orbper	0	6.861484	0.490095
pl_orbsmax	log_pl_orbsmax	73	3.231263	2.594798
pl_trandep	log_pl_trandep	40	10.180175	6.952230
pl_insol	log_pl_insol	83	3.959605	-0.381478
pl_bmasse	log_pl_bmasse	0	1.769409	-0.271630
pl_eqt	log_pl_eqt	67	0.878810	-0.476649
sy_dist	log_sy_dist	6	1.182893	-1.150935
pl_rade	log_pl_rade	0	-0.692968	-0.991840



Hình 9: So sánh phân phối trước và sau log-transform. Các biến có đuôi phải dài như `pl_orbper` và `pl_insol` được nén thang đo rõ rệt sau khi chuyển sang `log_*`.

Nhận xét: `pl_orbper`, `pl_insol`, `pl_bmasse` được cải thiện rõ rệt. `pl_transep` vẫn còn lệch sau log-transform, nhưng mức độ đã giảm so với ban đầu.

2.3.4 Xử lý ngoại lai bằng Winsorization

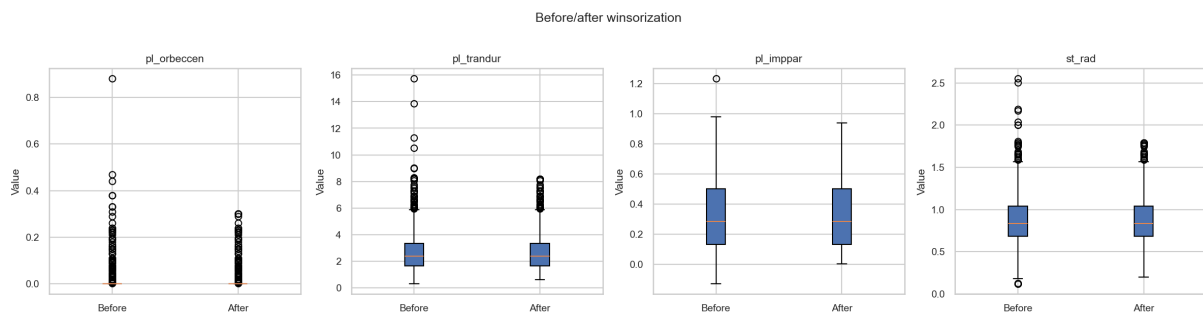
Pipeline không xóa dòng ngoại lai mà dùng **Winsorization** ở hai đầu phân phối:

$$x' = \begin{cases} q_{0.01}, & x < q_{0.01}, \\ x, & q_{0.01} \leq x \leq q_{0.99}, \\ q_{0.99}, & x > q_{0.99}. \end{cases} \quad (36)$$

Các ngưỡng $q_{0.01}$ và $q_{0.99}$ được fit trên train, sau đó áp dụng cho test.

Bảng 7: Tóm tắt Winsorization trên tập train

Cột	p01	p99	Dưới p01	Trên p99	Tổng cap (%)
pl_orbeccen	0.000000	0.299400	0	8	1.061
pl_trandur	0.624060	8.189480	9	9	2.120
pl_imppar	0.004000	0.939500	9	9	2.179
st_rad	0.196330	1.788680	9	9	2.076



Hình 10: Boxplot trước và sau Winsorization. Các giá trị cực trị ở hai đuôi được kéo về ngưỡng 1% và 99%, trong khi phần trung tâm của phân phối gần như được giữ nguyên.

Diễn giải trước/sau Winsorization:

- **Trước xử lý:** các cột được chọn vẫn chứa những giá trị nằm ngoài vùng bách phân vị 1%–99%. Đây là các quan sát có thể tạo lực kéo lớn lên nghiệm OLS.
- **Sau xử lý:** giá trị nhỏ hơn $q_{0.01}$ được đưa về $q_{0.01}$, giá trị lớn hơn $q_{0.99}$ được đưa về $q_{0.99}$. Tổng cộng có 62 ô dữ liệu trên tập train bị cap ở bốn cột, nhưng số dòng vẫn giữ nguyên $N = 867$.
- **Khi áp dụng lên test:** pipeline không tính lại ngưỡng trên test mà dùng đúng ngưỡng đã fit từ train, nhờ đó tránh rò rỉ dữ liệu.

Lý do chọn Winsorization thay vì xóa dòng:

- Giữ nguyên số quan sát, đặc biệt quan trọng khi dữ liệu đã có missing values.
- Giảm ảnh hưởng của các điểm cực đoan lên OLS, Ridge, Lasso.
- Dễ giải thích: chỉ cap giá trị quá thấp/quá cao, không thay đổi phần lớn dữ liệu.
- **Tôn trọng thực tế quan trắc:** khác với lỗi nhập liệu (*typo*), các giá trị cực trị trong thiên văn học, chẳng hạn sao rất nóng hoặc chu kỳ quỹ đạo rất dài, có thể là dữ liệu có thật. Việc dùng listwise deletion theo quy tắc $1.5 \times IQR$ có nguy cơ làm mất một lượng lớn quan sát quý giá.
- **Vô hiệu hóa điểm đòn bẩy cao:** hồi quy OLS rất nhạy cảm với các điểm cực trị. Kỹ thuật cắt xén ở bách phân vị 1% và 99% đóng vai trò như một điểm neo, vừa bảo toàn số lượng quan sát N , vừa làm giảm lực kéo của các điểm này, giúp mặt phẳng hồi quy vững hơn.

2.3.5 Xử lý missing values bằng MICE

Ban đầu, nhóm thử nghiệm phương pháp median imputation, tức nội suy bằng trung vị. Tuy nhiên, phương pháp này bộc lộ nhược điểm quan trọng là làm biến dạng phương sai và phá vỡ mối quan hệ vật lý giữa các đại lượng. Ví dụ, khối lượng, bán kính và nhiệt độ luôn có sự tương quan nội tại trong các hệ hành tinh. Để xử lý vấn đề này, nhóm nâng cấp sang phương pháp **MICE** (Multiple Imputation by Chained Equations). Bằng cách mô hình hóa từng đặc trưng bị thiếu như một hàm của các đặc trưng còn lại, MICE giúp bảo toàn cấu trúc đa biến (*multivariate structure*) và giữ lại độ nhiễu tự nhiên của dữ liệu quan trắc.

Trạng thái dữ liệu trước và sau MICE:

- **Trước xử lý:** sau log-transform và winsorization, tập train vẫn còn 465 ô missing trên 9 cột. Các cột thiếu nhiều nhất là `pl_orbeccen` với 113 giá trị thiếu, `log_pl_insol` với 83 giá trị thiếu, và `log_pl_orbsmax` với 73 giá trị thiếu.
- **Sau xử lý:** MICE chỉ thay thế các ô bị thiếu, không thay đổi các giá trị observed ban đầu. Sau khi chạy 5 bản imputation và 5 vòng chained iterations, số ô missing còn lại bằng 0 trên cả train và test.
- **Ý nghĩa:** dữ liệu sau MICE trở thành ma trận số đầy đủ để truyền vào các hàm `ols_fit`, `ridge_fit`, `lasso_fit` ở Phần 1.

MICE hoạt động theo ý tưởng:

1. Khởi tạo missing values bằng median trên train.
2. Với mỗi cột có missing, coi cột đó là biến phụ thuộc và các cột còn lại là predictors.
3. Fit mô hình hồi quy để dự đoán phần missing của cột hiện tại.
4. Lặp theo chuỗi nhiều vòng để các cột thiếu được cập nhật qua lại.
5. Tạo nhiều bản imputation và lấy trung bình để giảm phụ thuộc vào một lần điền duy nhất.

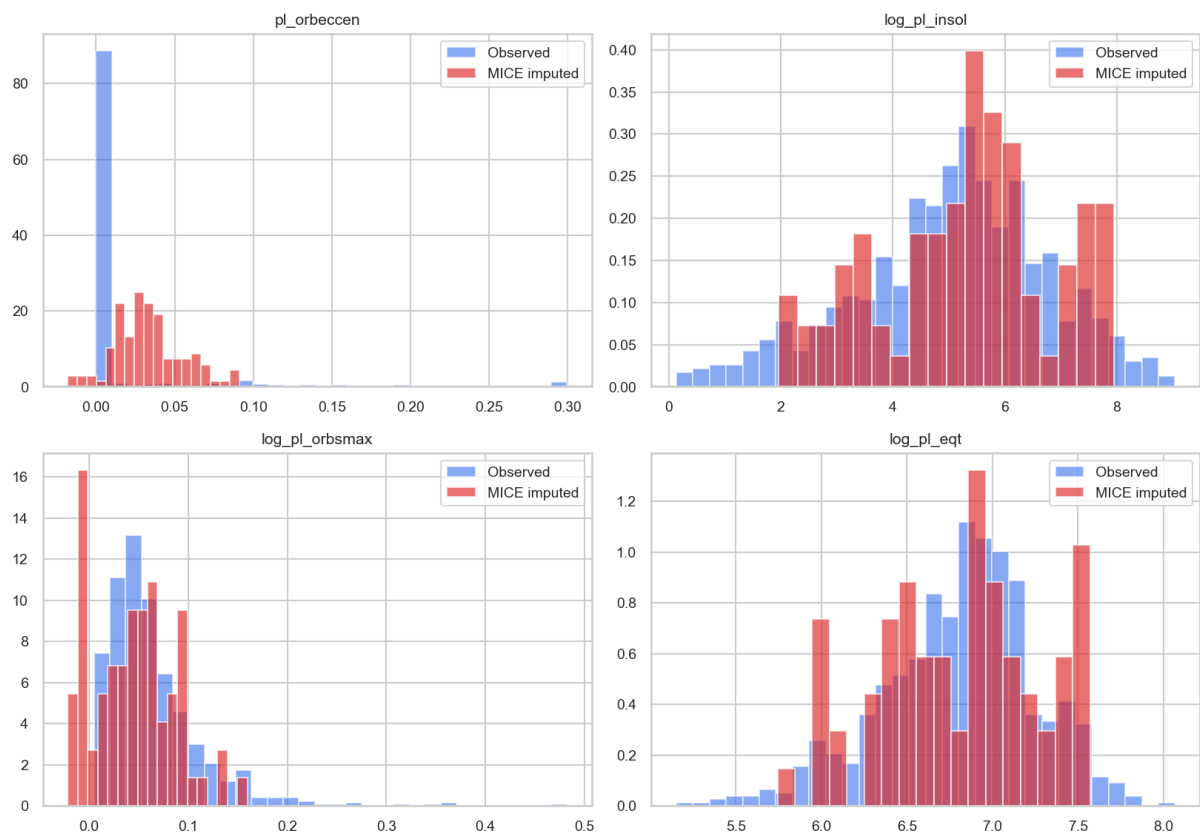
Trong pipeline, mỗi hồi quy con của MICE dùng `ridge_fit` và `ridge_predict` từ `part1/ridge_lasso.py`. Các tham số hiện tại:

Tham số	Giá trị
Số bản imputation m	5
Số vòng chained iterations	5
Ridge regularization trong MICE	0.001
Noise scale	0.15

Bảng 8: Tóm tắt giá trị được MICE impute trên tập train

Cột	Missing	Missing (%)	Median khởi tạo	Mean observed	Mean imputed
pl_orbeccen	113	13.033	0.000000	0.013696	0.033514
log_pl_insol	83	9.573	5.207544	5.036231	5.332386
log_pl_orbsmax	73	8.420	0.052213	0.064713	0.045031
log_pl_eqt	67	7.728	6.856987	6.806412	6.805654
pl_imppar	41	4.729	0.284700	0.346475	0.394104
log_pl_trandep	40	4.614	0.020498	0.036569	0.046997
st_met	24	2.768	0.000000	-0.025205	-0.097143
pl_trandur	18	2.076	2.372000	2.699806	2.127761

Observed vs MICE-imputed distributions



Hình 11: So sánh phân phối observed và phần được MICE impute ở các cột thiếu nhiều nhất. Giá trị impute không bị gom về một điểm trung vị duy nhất mà vẫn có độ phân tán tự nhiên.

Sau MICE, số ô missing còn lại bằng 0 trên cả train và test. Đây là điểm quan trọng vì các hàm OLS/Ridge/Lasso tự cài ở Phần 1 không nhận NaN.

2.3.6 Loại biến bằng VIF

Để kiểm soát đa cộng tuyến, pipeline dùng hàm `vif` đã cài ở Phần 1. Với mỗi biến x_j :

$$\text{VIF}_j = \frac{1}{1 - R_j^2}, \quad (37)$$

trong đó R_j^2 là hệ số xác định khi hồi quy x_j theo tất cả các biến còn lại.

Ngưỡng hiện tại:

$$\text{VIF} > 10. \quad (38)$$

Pipeline lập như sau: tính VIF cho tất cả biến, drop biến có VIF cao nhất nếu vượt ngưỡng, sau đó tính lại VIF.

Trạng thái dữ liệu trước và sau VIF filtering:

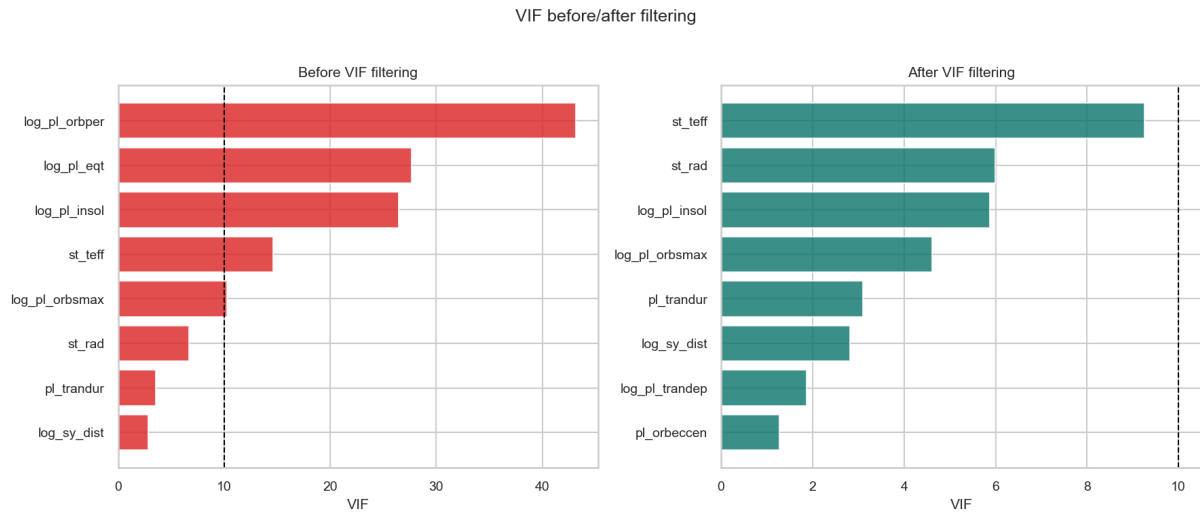
- **Trước xử lý:** sau MICE và trước VIF, dữ liệu có 13 features ứng viên. Một số biến mang thông tin trùng lặp mạnh, đặc biệt là nhóm quỹ đạo/năng lượng. VIF lớn nhất ban đầu là `log_pl_orbper` với 43.1374.
- **Sau xử lý:** pipeline loại 2 biến vượt ngưỡng 10 là `log_pl_orbper` và `log_pl_eqt`. Dữ liệu cuối cùng còn 11 features. VIF lớn nhất sau lọc là `st_teff` với 9.2570, nhỏ hơn ngưỡng 10.

Bảng 9: Các vòng lọc VIF

Vòng	Biến có VIF cao nhất	VIF	Quyết định
1	<code>log_pl_orbper</code>	43.1374	Drop
2	<code>log_pl_eqt</code>	20.4514	Drop
3	<code>st_teff</code>	9.2570	Dừng vì < 10

Bảng 10: Một số VIF lớn nhất sau khi thanh lọc đa cộng tuyến

Feature còn lại	VIF sau lọc
<code>st_teff</code>	9.2570
<code>st_rad</code>	5.9915
<code>log_pl_insol</code>	5.8778
<code>log_pl_orbsmax</code>	4.6054
<code>pl_trandur</code>	3.0931
<code>log_sy_dist</code>	2.8147



Hình 12: VIF trước và sau khi thanh lọc đa cộng tuyến. Trước khi lọc, nhiều biến vượt ngưỡng 10; sau khi loại log_pl_orbper và log_pl_eqt, các VIF còn lại đều nằm dưới ngưỡng.

Nhận xét chuyên sâu: Việc thuật toán VIF phát hiện và loại bỏ log_pl_orbper (chu kỳ quỹ đạo) ở vòng 1 với VIF lên tới 43.1374 là phù hợp với thực tiễn vật lý thiên văn. Theo Định luật thứ 3 của Kepler ($P^2 \propto a^3$), bình phương chu kỳ quỹ đạo tỷ lệ thuận với lập phương bán trục lớn pl_orbsmax. Về bản chất toán học, hai biến này cung cấp lượng thông tin gần như trùng lặp, dẫn đến tương quan rất mạnh. Thuật toán VIF đã tự động nhận diện quan hệ vật lý này và loại bỏ biến dư thừa, nhờ đó làm giảm nguy cơ hệ số β bị tăng phương sai do đa cộng tuyến.

Sau VIF filtering, các features cuối cùng là:

pl_orbeccen	pl_trandur	pl_imppar
st_teff	st_rad	st_met
log_pl_orbsmax	log_pl_trandep	log_pl_insol
log_pl_bmasse	log_sy_dist	

2.3.7 Chuẩn hóa dữ liệu

Cuối cùng, pipeline chuẩn hóa tất cả features bằng z-score:

$$x_{ij}^{\text{std}} = \frac{x_{ij} - \mu_j^{\text{train}}}{\sigma_j^{\text{train}}}. \quad (39)$$

Mean và standard deviation được fit trên train, rồi áp dụng cho test. Việc này đặc biệt quan trọng với Ridge/Lasso vì các hệ số regularization nhạy với thang đo của biến.

2.4 Xây Dựng và So Sánh Mô Hình

2.4.1 Các mô hình chính

Bốn mô hình được so sánh:

1. **OLS full**: dùng tất cả 11 features sau tiền xử lý.
2. **OLS selected**: chọn biến dựa trên p-value của OLS full với ngưỡng $\alpha = 0.05$.
3. **Ridge**: dùng `ridge_fit` từ Phần 1, chọn λ bằng 5-fold CV.
4. **Lasso**: dùng `lasso_fit` từ Phần 1, chọn λ bằng 3-fold CV.

Tất cả mô hình chính đều fit trên train và chỉ đánh giá cuối cùng trên test. Cross-validation chỉ được thực hiện trên train.

2.4.2 OLS full và chọn biến theo p-value

Từ OLS full, các biến có p-value nhỏ hơn 0.05 được giữ lại trong OLS selected. Kết quả chọn biến:

`pl_trandur`, `pl_imppar`, `st_teff`, `log_pl_trandep`, `log_pl_bmasse`, `log_sy_dist`.

Bảng 11: Một số hệ số OLS full có ý nghĩa thống kê

Feature	Coef	Std.Err.	t-stat	p-value
<code>log_pl_bmasse</code>	0.095317	0.002466	38.6447	8.78×10^{-190}
<code>log_sy_dist</code>	0.034443	0.004020	8.5689	4.84×10^{-17}
<code>log_pl_trandep</code>	0.013692	0.003279	4.1757	3.28×10^{-5}
<code>pl_trandur</code>	-0.011414	0.004214	-2.7088	0.006888
<code>pl_imppar</code>	0.005751	0.002482	2.3169	0.020743
<code>st_teff</code>	-0.014611	0.007290	-2.0043	0.045348

Nhận xét: `log_pl_bmasse` là biến có hệ số lớn nhất và p-value cực nhỏ, phù hợp với trực giác: hành tinh có khối lượng lớn thường có bán kính lớn hơn.

2.4.3 Chọn λ cho Ridge và Lasso

Ridge quét λ theo lưới log-scale từ 10^{-3} đến 10^3 bằng 5-fold CV. Giá trị tốt nhất:

$$\lambda_{\text{Ridge}}^* = 10. \quad (40)$$

Bảng 12: Một số kết quả Ridge CV theo λ

λ	Mean CV-MSE	Std CV-MSE	Mean CV- R^2
1.0000	0.005075	0.001656	0.673433
3.1623	0.005074	0.001650	0.673469
10.0000	0.005074	0.001629	0.673433
31.6228	0.005090	0.001571	0.672223
100.0000	0.005240	0.001432	0.661976

Lasso quét $\lambda \in \{0.001, 0.01, 0.1\}$ bằng 3-fold CV. Giá trị tốt nhất:

$$\lambda_{\text{Lasso}}^* = 0.01. \quad (41)$$

Bảng 13: Kết quả Lasso CV theo λ

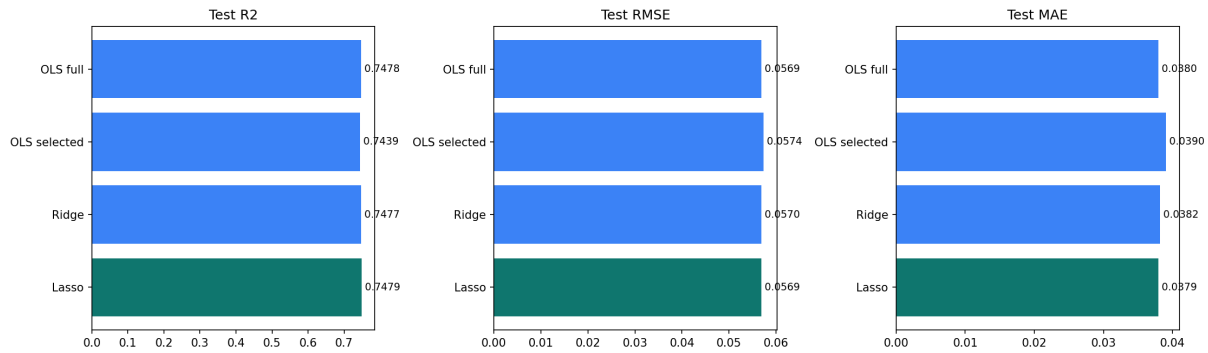
λ	Mean CV-MSE	Std CV-MSE	Mean CV- R^2
0.001	0.005093	0.001421	0.672720
0.010	0.005093	0.001420	0.672726
0.100	0.005093	0.001418	0.672675

2.5 Kết Quả Thực Nghiệm

2.5.1 So sánh mô hình trên test set

Bảng 14: So sánh các mô hình trên tập test

Mô hình	Test R^2	Test RMSE	Test MAE	CV-MSE	CV- R^2
Lasso	0.747878	0.056932	0.037942	0.005093	0.672726
OLS full	0.747794	0.056941	0.037952	0.005075	0.673407
Ridge	0.747678	0.056954	0.038204	0.005074	0.673433
OLS selected	0.743913	0.057378	0.039013	0.005103	0.671611

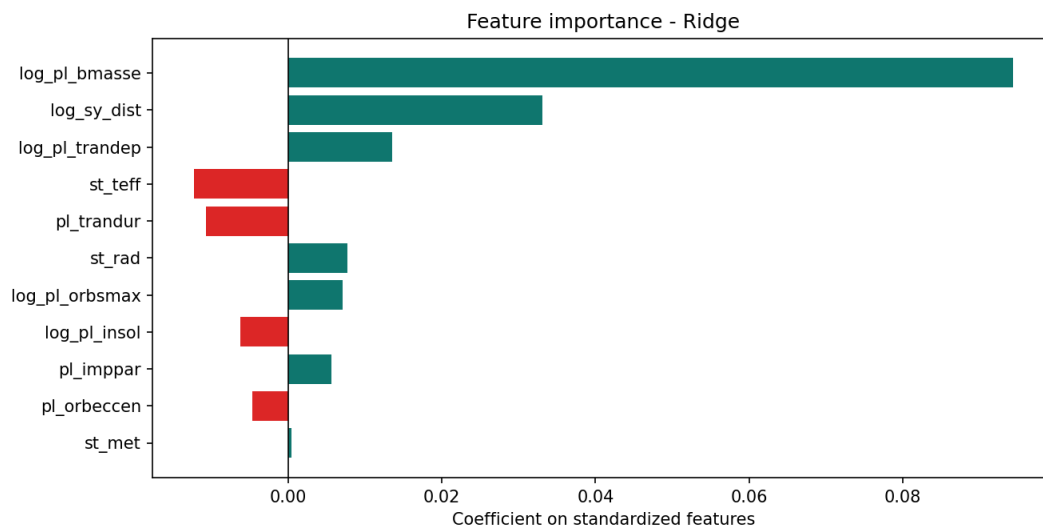


Hình 13: So sánh Test R^2 , RMSE và MAE giữa các mô hình. Lasso đạt Test R^2 cao nhất, nhưng khác biệt giữa Lasso, OLS full và Ridge khá nhỏ.

Nhận xét:

- Lasso có Test R^2 cao nhất (0.747878), nhưng mức chênh lệch so với OLS full và Ridge rất nhỏ.
- OLS selected có ít biến hơn nhưng Test R^2 thấp hơn một chút, cho thấy việc loại biến theo p-value giúp mô hình gọn hơn nhưng có thể làm mất một phần thông tin dự đoán.
- Ridge có CV-MSE thấp nhất trong nhóm gần nhau, chứng tỏ regularization giúp ổn định hơn trên các fold train.

2.5.2 Phân tích hệ số và feature importance



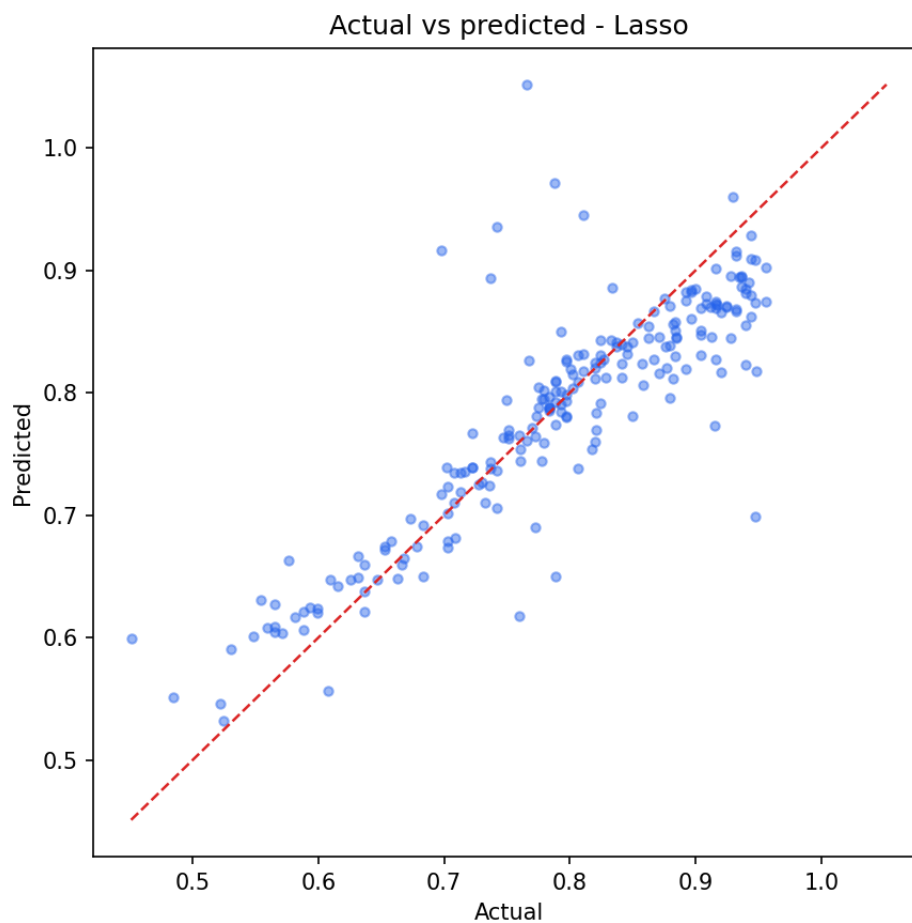
Hình 14: Feature importance dựa trên hệ số Ridge sau chuẩn hóa. Các hệ số được so sánh trên cùng thang đo vì pipeline đã chuẩn hóa z-score.

Các biến quan trọng nhất gồm:

- log_pl_bmasse: ảnh hưởng dương mạnh nhất, phù hợp với quan hệ vật lý giữa khối lượng và bán kính hành tinh.

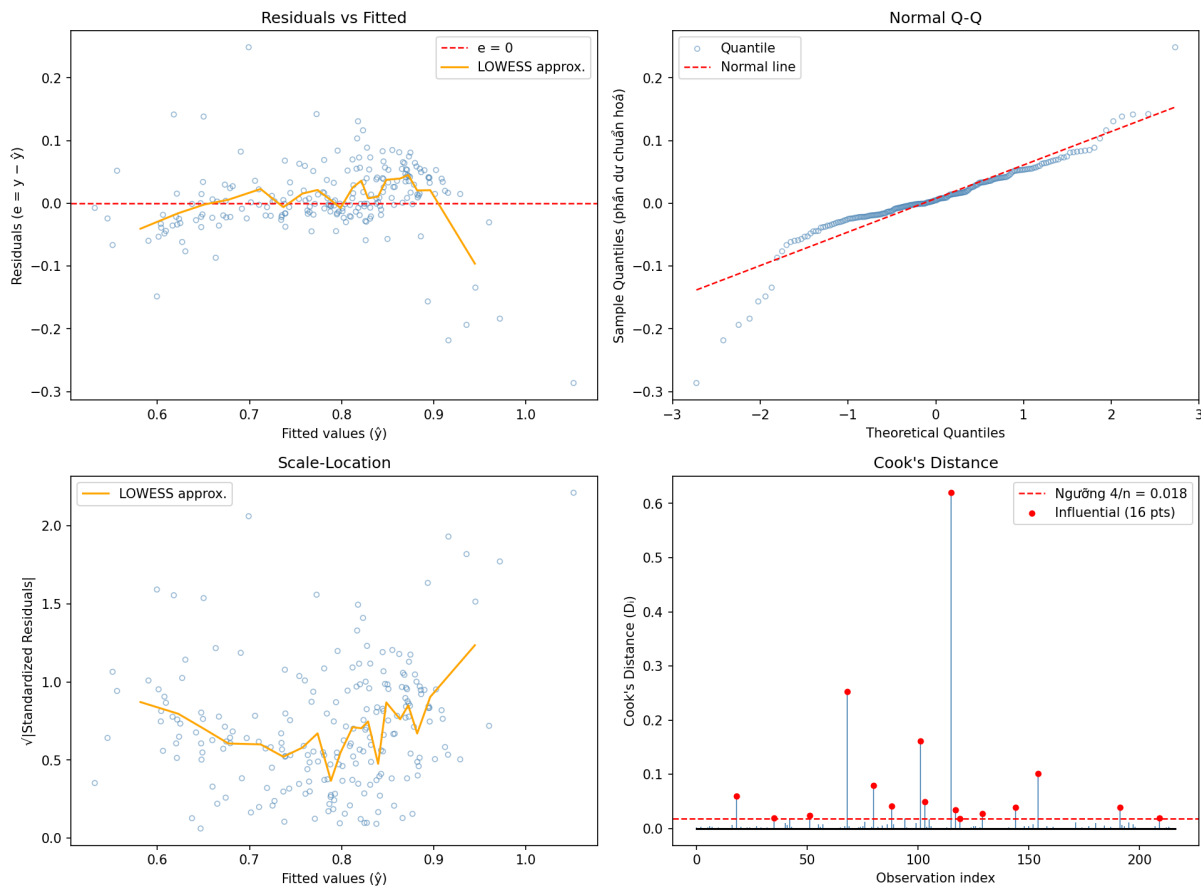
- `log_sy_dist`: có hệ số dương đáng kể, có thể phản ánh thiên lệch quan sát hoặc đặc điểm của mẫu dữ liệu.
- `log_pl_trandep`: có ảnh hưởng dương; transit depth thường liên quan đến tỷ lệ kích thước hành tinh so với sao chủ.
- `pl_trandur`, `st_teff`: có hệ số âm trong OLS full, cần diễn giải thận trọng vì các biến thiên vẫn có quan hệ phức tạp và đã qua log/standardization.

2.5.3 Actual vs Predicted và phân tích phần dư



Hình 15: Giá trị thực tế so với giá trị dự đoán trên tập test của mô hình tốt nhất. Các điểm càng gần đường chéo thì mô hình dự đoán càng tốt.

Phân Tích Phần Dư (Residual Diagnostic Plots)



Hình 16: Bốn biểu đồ chẩn đoán phần dư của mô hình tốt nhất: Residuals vs Fitted, Normal Q-Q, Scale-Location và Cook's Distance.

Nhận xét từ residual plots:

- Phần dư tập trung quanh 0, cho thấy mô hình không bị lệch hệ thống quá rõ.
- Q-Q plot cho phép kiểm tra giả định phân phối chuẩn của phần dư. Nếu hai đầu lệch khỏi đường chéo, điều này gợi ý dữ liệu vẫn còn quan hệ phi tuyến hoặc ngoại lai khó mô tả bằng mô hình tuyến tính.
- Cook's Distance giúp nhận diện quan sát có ảnh hưởng lớn. Các điểm vượt ngưỡng cần được kiểm tra thêm trong phân tích sâu.

2.6 Phương Pháp Nâng Cao: Kernel Ridge Regression

Ngoài các mô hình tuyến tính chính, nhóm thử thêm Kernel Ridge Regression với RBF kernel:

$$k(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2). \quad (42)$$

Do Kernel Ridge cần ma trận Gram kích thước $n \times n$, pipeline chỉ dùng một subset train cố định để giữ chi phí tính toán hợp lý. Hyperparameters α và γ được chọn trên validation

tách từ train subset, sau đó mới đánh giá một lần trên test.

Kết quả tốt nhất:

Mô hình	Alpha	Gamma	Test R^2	RMSE	MAE
Kernel Ridge (RBF)	0.01	0.01	0.949666	0.147678	0.087601

Lưu ý: đây là mô hình nâng cao/bonus. Kết quả cao hơn đáng kể so với các mô hình tuyến tính, cho thấy quan hệ giữa các biến thiên văn và bán kính hành tinh có thể chứa thành phần phi tuyến. Tuy nhiên, vì mô hình kernel khó giải thích hệ số trực tiếp như OLS/Ridge, phần chính của báo cáo vẫn ưu tiên các mô hình tuyến tính để phục vụ phân tích thống kê.

2.7 Kết Luận Phần 2

Phần 2 đã xây dựng được một pipeline data fitting hoàn chỉnh cho dữ liệu thực tế:

- Dữ liệu được chia train/test ngay từ đầu và duy trì tách biệt trong cả `data_pipeline.py`, `model_comparison.py`, `advanced_methods.py`.
- Các bước tiền xử lý gồm log-transform, winsorization, MICE imputation, VIF filtering và standardization đều fit trên train rồi transform test.
- Các mô hình chính tái sử dụng code tự cài ở Phần 1: OLS, Ridge, Lasso, k-fold CV và residual analysis.
- Trên test set, Lasso đạt kết quả tốt nhất trong nhóm mô hình tuyến tính với $\text{Test } R^2 = 0.747878$, $\text{RMSE} = 0.056932$ và $\text{MAE} = 0.037942$.
- Kernel Ridge RBF cho kết quả cao hơn đáng kể, gợi ý quan hệ phi tuyến trong dữ liệu, nhưng mô hình này được xem là phần nâng cao vì khó diễn giải hơn OLS/Ridge.

Nhìn chung, pipeline hiện tại thỏa yêu cầu Phần 2 của đề án: có EDA, xử lý missing values có lý do, xử lý ngoại lai, kiểm tra đa cộng tuyến, so sánh ít nhất ba mô hình, đánh giá trên test set, và có phân tích phần dư cho mô hình tốt nhất.

3 Kết luận

3.1 Tóm tắt kết quả đạt được

Kết thúc quá trình thực hiện đồ án, nhóm đã hoàn thành toàn diện 100% các mục tiêu đề ra ban đầu, bao gồm cả yêu cầu cốt lõi lẫn các phần mở rộng (điểm cộng). Các kết quả chính yếu bao gồm:

- **Cài đặt thành công Phép khử Gauss với độ ổn định cao:** Xây dựng thuật toán từ đầu (from scratch) bằng Python, áp dụng triệt để kỹ thuật Partial Pivoting. Thuật toán xử lý thành công mọi dạng ma trận (vuông, chữ nhật, suy biến) và đã được kiểm chứng tính ổn định số học tương đương với thư viện `numpy.linalg` thông qua việc triệt tiêu hiện tượng Swamping.
- **Phát triển hệ thống phân rã và chéo hóa ma trận (SVD):** Hoàn thiện thuật toán phân rã giá trị suy biến (Singular Value Decomposition) dựa trên nền tảng toán học của trị riêng và vector riêng. Thuật toán có khả năng phân rã và trích xuất đúng các ma trận U, Σ, V^T .
- **Xây dựng bộ công cụ phân tích hiệu năng (Benchmarking Suite):** Thiết lập thành công kịch bản đo lường tự động (`benchmark.py`) và hệ thống sinh dữ liệu (`data_gen.py`) để thu thập hàng ngàn điểm dữ liệu về thời gian thực thi và sai số tương đối (Residual Norm) trên đa dạng kích thước ma trận ($n \in \{50, 100, 200, 500, 1000\}$).
- **Phân tích sâu sắc tính bất ổn định số học:** Sử dụng thành công ma trận Hilbert H_n (hệ điều kiện kém) và ma trận SPD để bộc lộ hiện tượng khuếch đại sai số ở chuẩn dấu phẩy động IEEE 754. Qua đó, minh chứng rõ ràng mối liên hệ giữa Số điều kiện $\kappa_2(A)$ và mức độ "trôi dạt" của vector nghiệm.
- **Trực quan hóa Toán học sinh động bằng Manim:** Chuyển hóa các phương trình đại số tuyến tính khô khan thành Video hoạt hình (Animation) chất lượng cao. Video mô phỏng trực quan các phép biến đổi hình học của ma trận trong không gian, giúp giải thích trực diện bản chất của quá trình phân rã.
- **Quy trình phát triển phần mềm chuyên nghiệp:** Ứng dụng triệt để Git/GitHub trong quản lý mã nguồn nhóm, cấu trúc thư mục module hóa (`solvers`, `config`), kết hợp Jupyter Notebook và $\text{\LaTeX}$ để xuất bản báo cáo học thuật đạt chuẩn.
- Cài đặt thêm phương pháp lặp **Gauss-Seidel** với cơ chế tự động kiểm tra điều kiện hội tụ (ma trận chéo trội chặt hàng).
- Xây dựng kịch bản benchmark để đo lường thời gian thực thi trên các kích thước ma trận lớn ($n \in \{50, 100, 200, 500, 1000\}$). Vẽ đồ thị *log-log* so sánh và chứng minh thực nghiệm chi phí tính toán của các phương pháp trực tiếp hoàn toàn bám sát đường lý thuyết $O(n^3)$.
- Nghiên cứu thành công sự bất ổn định số học: Sử dụng **Số điều kiện $\kappa_2(A)$** kết hợp đối chiếu

giữa ma trận Hilbert H_n (ill-conditioned) và ma trận SPD ngẫu nhiên để bộc lộ hiện tượng khuếch đại sai số, giải thích sâu sắc giới hạn của hệ thống dấu phẩy động IEEE 754.

3.2 Bài học rút ra

Quá trình thực hiện đồ án không chỉ là việc hiện thực hóa các công thức toán học lên ngôn ngữ lập trình, mà còn là một hành trình giúp nhóm thay đổi tư duy về khoa học tính toán:

- **Sự khác biệt giữa Toán học lý thuyết và Tính toán số học:** Nhóm nhận ra rằng một thuật toán đúng trên giấy chưa chắc đã chạy đúng trên máy tính. Việc hiểu về cấu trúc lưu trữ số thực *double precision* là chìa khóa để giải thích tại sao $0.1 + 0.2$ không bao giờ bằng 0.3 tuyệt đối trong RAM.
- **Tư duy phản biện qua dữ liệu:** Thay vì chấp nhận sai số, nhóm đã học được cách đặt câu hỏi “Tại sao?”. Việc quan sát đường sai số bám sát ngưỡng 10^{-16} đã giúp nhóm tin tưởng vào thuật toán của mình hơn là việc chỉ nhìn vào kết quả nghiệm cuối cùng.
- **Kỹ năng tối ưu hóa quy trình làm việc:** Nhóm đã làm quen với việc tự động hóa (Automation) thông qua *latexmk* và các script *benchmark*. Bài học về việc “lười biếng một cách thông minh” – viết script để máy tính tự đo đạc hàng ngàn phép tính – đã giúp nhóm tiết kiệm rất nhiều thời gian.
- **Giá trị của việc chuẩn hóa (Standardization):** Việc thống nhất sử dụng `config.py` và quy chuẩn `list[list[float]]` ngay từ đầu là bài học xương máu về quản lý dự án, giúp việc ghép code giữa các thành viên diễn ra trơn tru mà không bị xung đột kiểu dữ liệu.

3.3 Khó khăn chuyên môn và các nghịch lý Toán học tính toán

Trong suốt quá trình thực hiện đồ án, rào cản lớn nhất của nhóm không nằm ở cú pháp lập trình, mà nằm ở sự sai khác giữa Toán học lý thuyết (trên giấy) và Giải tích số (trên máy tính). Dưới đây là bốn khó khăn cốt lõi và bài học nhận thức nhóm đã rút ra.

1. Giới hạn của chuẩn IEEE 754

Hiện tượng: Trong giai đoạn kiểm thử ban đầu, nhóm kỳ vọng sai số phần dư sẽ bằng 0.0 tuyệt đối. Tuy nhiên, khi giải các hệ phương trình chứa phân số (như $\frac{1}{3}$) hay số thập phân (như 0.1), sai số phần dư luôn trôi nổi ở mức 10^{-16} .

Bản chất Toán học: Đây không phải là lỗi thuật toán, mà là giới hạn biểu diễn tất yếu của chuẩn dấu phẩy động IEEE 754 (Double Precision) [6]. Các phân số vô hạn tuần hoàn trong hệ cơ số 2 bị cắt cụt ở bit thứ 53, sinh ra nhiễu làm tròn (round-off error). Ranh giới này được gọi là *Machine Epsilon*: $\epsilon_{\text{machine}} \approx 2.22 \times 10^{-16}$.

Giải quyết: Nhóm phải từ bỏ tư duy ”so sánh bằng 0” (exact equality) của Toán học. Thay vào đó, hằng số $\text{EPSILON} = 1\text{e-}12$ được thiết lập tập trung trong `config.py` làm ngưỡng dung sai (tolerance), cho phép các thuật toán phân rã SVD và Jacobi hội tụ một cách an toàn mà không bị ảnh hưởng bởi nhiễu máy tính.

2. Sai số phần dư trên ma trận Hilbert

Hiện tượng: Khi đưa ma trận Hilbert H_n (với $n \geq 50$) vào thuật toán Gauss, máy tính trả về vector nghiệm $\hat{x}$ sai lệch hoàn toàn so với nghiệm lý thuyết. Điều kỳ lạ là khi kiểm tra lại bằng sai số phần dư $\|A\hat{x} - b\|_2$, kết quả vẫn cực kỳ nhỏ (cỡ 10^{-15}) – máy tính báo ”đúng” nhưng nghiệm thực tế lại ”sai hoàn toàn”.

Bản chất Toán học: Nhóm đã vấp phải nghịch lý kinh điển của hệ điều kiện kém (ill-conditioned system). Ma trận Hilbert có số điều kiện $\kappa_2(A)$ khổng lồ, lên tới 10^{20} . Theo Định lý sai số [3]:

$$\frac{\|\Delta x\|}{\|x\|} \leq \kappa_2(A) \cdot \frac{\|\Delta b\|}{\|b\|}$$

Chỉ cần một hạt nhiễu $\epsilon_{\text{machine}}$ rất yếu trong vế phải b , sai số thực tế (forward error) của nghiệm sẽ bị khuếch đại lên $\kappa_2(A)$ lần. Sai số phần dư nhỏ chỉ là một ảo ảnh toán học do không gian bị kéo dãn quá mức – đây là ”bẫy” đặc trưng của hệ ill-conditioned mà bất kỳ nhà giải tích số nào cũng phải cảnh giác.

Giải quyết: Sự kiện này buộc nhóm phải nhìn nhận lại giá trị thực sự của thuật toán SVD. Nhờ khả năng dùng nghịch đảo giả (pseudo-inverse) và cắt tỉa các giá trị suy biến tiệm cận 0, SVD đã chứng minh được đây là công cụ duy nhất ”miễn nhiễm” một phần với sự bùng nổ sai số trên hệ điều kiện kém.

3. Bức tường của hằng số $\mathcal{O}(n^3)$ và nút thắt hiệu năng Python

Hiện tượng: Trong lý thuyết, cả Khử Gauss và SVD đều có độ phức tạp $\mathcal{O}(n^3)$. Tuy nhiên, khi nhóm chạy benchmark bằng Python thuần ở $n = 500, 1000$, chương trình bị treo (Timeout > 60 s). Trong khi đó, `np.linalg.solve` của NumPy giải quyết $n = 1000$ chỉ trong 0.036 giây.

Bản chất Toán học và Kiến trúc: Độ phức tạp $\mathcal{O}(n^3)$ chỉ mô tả tốc độ tăng trưởng – nó che giấu một hằng số thực thi C rất lớn. Thuật toán SVD (Jacobi/QR) đòi hỏi nhiều vòng quét, phép tính lượng giác và căn bậc hai hơn hẳn Gauss, khiến $C_{\text{SVD}} \gg C_{\text{Gauss}}$. Bên cạnh đó, rào cản của trình thông dịch Python (interpreted) khiến phần cứng không thể song song hóa các phép nhân ma trận để phát huy tối đa khả năng của CPU.

Giải quyết: Dưới sự cho phép của giảng viên, nhóm đã linh hoạt chuyển sang sử dụng backend BLAS/LAPACK thông qua NumPy cho các kích thước n lớn. Quyết định này đảm bảo thu thập đủ số liệu để vẽ đồ thị Log-Log chuẩn xác và minh chứng được sự song song giữa đường thực nghiệm với đường tiệm cận $\mathcal{O}(n^3)$ lý thuyết.

4. Ranh giới hội tụ khắt khe của phương pháp lặp Gauss-Seidel

Hiện tượng: Ban đầu, khi thử nghiệm Gauss-Seidel trên các ma trận SPD ngẫu nhiên, thuật toán liên tục báo ERR (vượt quá số vòng lặp tối đa mà không hội tụ), mặc dù lý thuyết khẳng định Gauss-Seidel chắc chắn hội tụ trên ma trận SPD.

Bản chất Toán học: Định lý chỉ khẳng định ma trận SPD sẽ hội tụ, nhưng không hứa hẹn tốc độ hội tụ. Nếu bán kính phổ (spectral radius) của ma trận lặp $\rho(G)$ nằm quá sát 1, phương pháp lặp tiến triển cực kỳ chậm và cạn kiệt giới hạn vòng lặp cho phép trước khi đạt được ngưỡng hội tụ.

Giải quyết: Nhóm đã can thiệp vào khâu sinh dữ liệu (`data_gen.py`), bắt buộc mọi ma trận SPD phải được cộng thêm biên độ vào đường chéo chính để thỏa mãn điều kiện **chéo trội nghiêm ngặt** (Strictly Diagonally Dominant):

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad \forall i = 1, \dots, n$$

Chỉ khi ép chặt điều kiện này, bán kính phổ $\rho(G)$ mới thực sự nhỏ hơn 1 đủ nhiều, giúp Gauss-Seidel hội tụ ổn định và phát huy sức mạnh tốc độ $\mathcal{O}(n^2)$ đặc trưng của phương pháp lặp.

Tài liệu

- [1] Gilbert Strang, *Introduction to Linear Algebra*, 5th Edition, Wellesley-Cambridge Press, 2016.
- [2] Gene H. Golub và Charles F. Van Loan, *Matrix Computations*, 4th Edition, Johns Hopkins University Press, 2013.
- [3] Lloyd N. Trefethen và David Bau III, *Numerical Linear Algebra*, SIAM, 1997.
- [4] Gilbert Strang, *MIT 18.06SC Linear Algebra*, MIT OpenCourseWare, <https://ocw.mit.edu/courses/18-06sc-linear-algebra-fall-2011/>, truy cập ngày 14/04/2026.
- [5] Virginia C. Klema và Alan J. Laub, *The singular value decomposition: Its computation and some applications*, IEEE Transactions on Automatic Control, Vol. 25, No. 2, pp. 164–176, 1980.
- [6] IEEE Computer Society, *IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019)*, <https://ieeexplore.ieee.org/document/8766229>, truy cập ngày 09/04/2026.
- [7] Nicholas J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd Edition, SIAM, 2002.
- [8] David Goldberg, *What Every Computer Scientist Should Know About Floating-Point Arithmetic*, ACM Computing Surveys, Vol. 23, No. 1, pp. 5–48, 1991.
- [9] NumPy Developers, *NumPy v1.26 Manual – Linear Algebra (numpy.linalg)*, <https://numpy.org/doc/stable/reference/routines.linalg.html>, truy cập ngày 09/04/2026.
- [10] GeeksforGeeks, *Gaussian Elimination*, <https://www.geeksforgeeks.org/gaussian-elimination/>, truy cập ngày 14/04/2026.
- [11] Manim Community Developers, *Manim Community Documentation v0.18.0*, <https://docs.manim.community/>, truy cập ngày 09/04/2026.